

ELMAR  
CNC-Cutter

Autoren: Alexandra Roth 7025637  
David Atrops 7013979  
Dennis Smirnow 7023434  
Leon Ronge 7026641  
Samuel Drees 7026354

Studiengang: Maschinenbau und Design

Erstprüfer: Prof. Dr. Elmar Wings  
Malena Wilken

Abgabedatum: 12. Mai 2026



# Inhaltsverzeichnis

|                                                                                                   |           |
|---------------------------------------------------------------------------------------------------|-----------|
| <b>Inhaltsverzeichnis</b>                                                                         | <b>3</b>  |
| <b>Abbildungsverzeichnis</b>                                                                      | <b>7</b>  |
| <b>I. Software</b>                                                                                | <b>9</b>  |
| <b>1. Arduino IDE 2.3.x</b>                                                                       | <b>11</b> |
| 1.1. Beschreibung der Arduino IDE . . . . .                                                       | 11        |
| 1.2. Installation der Arduino IDE . . . . .                                                       | 12        |
| 1.2.1. Download der Arduino IDE . . . . .                                                         | 12        |
| 1.2.2. Zusammenfassung der Optionen . . . . .                                                     | 13        |
| 1.2.3. Installation der Arduino IDE für Windows . . . . .                                         | 13        |
| 1.3. Verknüpfen des Boards mit der IDE . . . . .                                                  | 15        |
| 1.3.1. Beispielcode „Blinkende LED“ . . . . .                                                     | 16        |
| 1.3.2. Installation der Arduino IDE für MacOS . . . . .                                           | 18        |
| 1.3.3. Bibliotheken installieren . . . . .                                                        | 23        |
| 1.3.4. Board Manager . . . . .                                                                    | 24        |
| 1.3.5. Beispielprogramm „Blink“ . . . . .                                                         | 26        |
| <b>2. Arduino IDE 2.3.6</b>                                                                       | <b>29</b> |
| 2.1. Arduino IDE Beschreibung . . . . .                                                           | 29        |
| 2.2. Installation der Arduino IDE . . . . .                                                       | 30        |
| 2.2.1. Download der Arduino IDE . . . . .                                                         | 30        |
| 2.2.2. Anforderungen Betriebssystem . . . . .                                                     | 31        |
| 2.2.3. Installation auf Windows . . . . .                                                         | 31        |
| 2.2.4. Installation (MacOS) . . . . .                                                             | 34        |
| 2.3. Übersicht der Eigenschaften . . . . .                                                        | 35        |
| 2.3.1. Sketchbook . . . . .                                                                       | 36        |
| 2.3.2. Boards Manager . . . . .                                                                   | 37        |
| 2.3.3. Library Manager . . . . .                                                                  | 37        |
| 2.4. Integration und Verwendung einer benutzerdefinierten Bibliothek in der Arduino IDE . . . . . | 38        |
| 2.4.1. Erstellen der Bibliotheksdateien . . . . .                                                 | 38        |
| 2.4.2. Installation und Integration der Arduino IDE . . . . .                                     | 38        |
| 2.4.3. Verwendung symbolischer Verknüpfungen zur Bibliotheksintegration . . . . .                 | 39        |
| 2.4.4. Windows-Tool <b>mklink</b> zum Erstellen symbolischer Verknüpfungen . . . . .              | 39        |
| 2.4.5. Überprüfen der Verfügbarkeit der Bibliothek in der Arduino IDE . . . . .                   | 40        |
| 2.4.6. Serieller Monitor . . . . .                                                                | 41        |
| 2.4.7. Serieller Plotter . . . . .                                                                | 41        |
| 2.5. Beispiele . . . . .                                                                          | 42        |
| 2.6. Debuggen . . . . .                                                                           | 42        |
| 2.7. Autovervollständigung . . . . .                                                              | 43        |
| 2.8. Bibliotheken und Modularität . . . . .                                                       | 44        |

|                                                                         |               |
|-------------------------------------------------------------------------|---------------|
| <b>3. Arduino Web Editor</b>                                            | <b>45</b>     |
| 3.1. Using the Arduino Web Editor . . . . .                             | 45            |
| 3.1.1. Using the online IDE . . . . .                                   | 45            |
| 3.2. Arduino Libraries . . . . .                                        | 46            |
| 3.3. Data Security . . . . .                                            | 47            |
| 3.3.1. Authentication and Authorization . . . . .                       | 47            |
| 3.3.2. Secure Code Execution . . . . .                                  | 47            |
| 3.3.3. Protection Against Malware . . . . .                             | 47            |
| 3.3.4. Privacy Policies . . . . .                                       | 47            |
| 3.3.5. Regular Updates and Maintenance . . . . .                        | 48            |
| 3.3.6. User Awareness . . . . .                                         | 48            |
| <br><b>II. Hardware</b>                                                 | <br><b>49</b> |
| <b>4. Arduino Uno</b>                                                   | <b>51</b>     |
| 4.1. Was ist ein Arduino? . . . . .                                     | 51            |
| 4.2. Arduino Uno . . . . .                                              | 51            |
| 4.3. Sensoren . . . . .                                                 | 51            |
| 4.3.1. Testtaster . . . . .                                             | 51            |
| 4.4. Module . . . . .                                                   | 51            |
| 4.5. PINs . . . . .                                                     | 51            |
| 4.6. Beispiel Code . . . . .                                            | 51            |
| 4.6.1. Manual . . . . .                                                 | 51            |
| 4.6.2. Code . . . . .                                                   | 51            |
| <br><b>5. CNCShieldV3</b>                                               | <br><b>53</b> |
| 5.1. CNC Shields . . . . .                                              | 53            |
| 5.2. CNCShieldV3 . . . . .                                              | 53            |
| 5.3. Specification . . . . .                                            | 53            |
| 5.4. Library . . . . .                                                  | 53            |
| 5.4.1. Description . . . . .                                            | 53            |
| 5.4.2. Installation . . . . .                                           | 53            |
| 5.4.3. Functions . . . . .                                              | 53            |
| 5.5. Example . . . . .                                                  | 53            |
| 5.5.1. Manual . . . . .                                                 | 53            |
| 5.5.2. Inside the Example . . . . .                                     | 53            |
| 5.5.3. Code . . . . .                                                   | 53            |
| 5.5.4. Files . . . . .                                                  | 53            |
| 5.6. Calibration . . . . .                                              | 53            |
| 5.7. Simple Code . . . . .                                              | 54            |
| 5.8. Simple Application . . . . .                                       | 54            |
| 5.9. Tests . . . . .                                                    | 54            |
| 5.9.1. Simple Function Test . . . . .                                   | 54            |
| 5.9.2. Test all Functions . . . . .                                     | 54            |
| 5.10. Further Readings . . . . .                                        | 54            |
| <br><b>6. Sensor: Endschalter</b>                                       | <br><b>55</b> |
| 6.1. Einleitung: Positionssensoren . . . . .                            | 55            |
| 6.2. Definition und Funktion: Elektromechanischer Endschalter . . . . . | 55            |
| 6.3. Specification . . . . .                                            | 55            |
| 6.4. Verwendung und Schaltfunktionen . . . . .                          | 56            |
| 6.5. Analyse: Vor- und Nachteile . . . . .                              | 57            |
| 6.6. Ausblick: Intelligente und vernetzte Sensorik . . . . .            | 58            |
| 6.7. Library . . . . .                                                  | 58            |

|                                                   |           |
|---------------------------------------------------|-----------|
| Inhaltsverzeichnis                                | 5         |
| 6.8. Beispiel . . . . .                           | 58        |
| 6.8.1. Manual . . . . .                           | 58        |
| 6.8.2. Erklärung des Beispiel-Codes . . . . .     | 58        |
| 6.9. Justage . . . . .                            | 59        |
| 6.10. Tests . . . . .                             | 59        |
| 6.10.1. Einfacher Funktions-Test . . . . .        | 59        |
| 6.10.2. Manual . . . . .                          | 60        |
| 6.10.3. Einfacher Test-Code . . . . .             | 60        |
| 6.11. Einfache Anwendung . . . . .                | 60        |
| 6.11.1. Manual . . . . .                          | 61        |
| 6.12. Weiterführende Literatur . . . . .          | 62        |
| <b>7. Druckknopf</b>                              | <b>63</b> |
| 7.1. General . . . . .                            | 63        |
| 7.2. Joy IT . . . . .                             | 63        |
| 7.3. Spezifikationen . . . . .                    | 63        |
| 7.3.1. Functions . . . . .                        | 63        |
| 7.4. Beispiel Code . . . . .                      | 63        |
| 7.4.1. Manual . . . . .                           | 63        |
| 7.4.2. Code . . . . .                             | 64        |
| 7.5. Einfacher Testcode . . . . .                 | 64        |
| 7.5.1. Manual . . . . .                           | 64        |
| 7.5.2. Code . . . . .                             | 65        |
| 7.6. Einfache Anwendung . . . . .                 | 65        |
| 7.6.1. Manual . . . . .                           | 65        |
| 7.6.2. Code . . . . .                             | 66        |
| 7.7. Further Readings . . . . .                   | 68        |
| <b>III. Domain Knowledge - Tools</b>              | <b>69</b> |
| <b>IV. Developmenmt</b>                           | <b>71</b> |
| <b>V. Monitoring</b>                              | <b>73</b> |
| <b>VI. Verification - Evaluation - Conclusion</b> | <b>75</b> |
| <b>VII.Anhang</b>                                 | <b>77</b> |
| <b>8. Bill of Material</b>                        | <b>79</b> |
| 8.1. Mechanische Komponenten . . . . .            | 79        |
| 8.2. Elektronische Komponenten . . . . .          | 80        |
| 8.3. Schrauben und Befestigungen . . . . .        | 80        |
| <b>Literaturverzeichnis</b>                       | <b>81</b> |
| <b>Literaturverzeichnis</b>                       | <b>81</b> |



# Abbildungsverzeichnis

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1.1. Menu Button . . . . .                                        | 11 |
| 1.2. Arduino IDE 2.3.3 Download . . . . .                         | 12 |
| 1.3. Zusammenfassung der Download-Optionen . . . . .              | 13 |
| 1.4. Downloadfenster Arduino IDE . . . . .                        | 13 |
| 1.5. Lizenzabkommen . . . . .                                     | 14 |
| 1.6. Installationsoption . . . . .                                | 14 |
| 1.7. Zielverzeichnisauswahl . . . . .                             | 15 |
| 1.8. Boardauswahl . . . . .                                       | 16 |
| 1.9. Portauswahl . . . . .                                        | 16 |
| 1.10. Aufruf der Arduino-Webseite über die Suchmaschine . . . . . | 18 |
| 1.11. Ergebnisse der Websuche zur Arduino IDE . . . . .           | 18 |
| 1.12. Download der Arduino IDE starten . . . . .                  | 19 |
| 1.13. Optional: Spendenaufruf beim Download . . . . .             | 19 |
| 1.14. Optional: Newsletter abonnieren . . . . .                   | 20 |
| 1.15. Download-Erlaubnis erteilen . . . . .                       | 20 |
| 1.16. Download-Fortschritt einsehen . . . . .                     | 21 |
| 1.17. Heruntergeladene Datei öffnen . . . . .                     | 21 |
| 1.18. Installation durch Ziehen in den Programme-Ordner . . . . . | 22 |
| 1.19. Arduino IDE starten . . . . .                               | 22 |
| 1.20. Sicherheitsabfrage bei erstmaligem Start . . . . .          | 23 |
| 1.21. Bibliotheken-Suchmaske der Arduino IDE . . . . .            | 24 |
| 1.22. Erfolgreiche Installation einer Bibliothek . . . . .        | 24 |
| 1.23. Der Board Manager in der Arduino IDE . . . . .              | 25 |
| 1.24. Board- und Portauswahl öffnen . . . . .                     | 25 |
| 1.25. Passenden Board und Port auswählen . . . . .                | 26 |
| 1.26. Beispielprogramme öffnen . . . . .                          | 26 |
| 1.27. Das geöffnete Beispielprogramm „Blink“ . . . . .            | 27 |
| 2.1. Arduino IDE 2.3.6 . . . . .                                  | 31 |
| 2.2. Arduino IDE Downloadoptionen . . . . .                       | 32 |
| 2.3. Lizenzbestätigung Installation . . . . .                     | 33 |
| 2.4. Wahl Zielverzeichnis . . . . .                               | 33 |
| 2.5. Startfenster Arduino-IDE . . . . .                           | 34 |
| 2.6. ArduinoIDE macOS . . . . .                                   | 35 |
| 2.7. Überblick . . . . .                                          | 36 |
| 2.8. Sketchbook . . . . .                                         | 36 |
| 2.9. Boards Manager . . . . .                                     | 37 |
| 2.10. Library Manager . . . . .                                   | 38 |
| 2.11. Eingabeaufforderung mit Administrator Rechten . . . . .     | 39 |
| 2.12. Eingabeaufforderung . . . . .                               | 40 |
| 2.13. Einbinden der Nano33BLESenseLED Bibliothek . . . . .        | 41 |
| 2.14. Serieller Monitor . . . . .                                 | 41 |
| 2.15. Beispiele . . . . .                                         | 42 |
| 2.16. Beispiel Debuggen . . . . .                                 | 43 |
| 2.17. Autovervollständigung . . . . .                             | 43 |
| 3.1. Arduino Web Editor . . . . .                                 | 45 |

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| 3.2. finding-an-example . . . . .                                         | 46 |
| 6.1. Darstellung des im Projekt verwendeten Endschalters . . . . .        | 56 |
| 6.2. Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO | 57 |



**Teil I.**

**Software**



# 1. Arduino IDE 2.3.x

Dieses Kapitel bietet eine umfassende Untersuchung der Arduino-IDE und behandelt ihre Merkmale, Schnittstellenoptionen und Funktionen. Es enthält eine detaillierte Erläuterung des Zwecks und der Benutzerfreundlichkeit jeder Komponente, eine schrittweise Installationsanleitung und Anweisungen zur Initialisierung und Konfiguration der Umgebung. Mit diesem grundlegenden Überblick wird der Leser in die Lage versetzt, die IDE effektiv für die Programmierung und Fehlerbehebung von Arduino-kompatibler Hardware zu nutzen.

## 1.1. Beschreibung der Arduino IDE

Die Arduino IDE ist eine offizielle Open-Source-Software zum Bearbeiten, Kompilieren und Hochladen von Code auf Arduino-Module. Sie ist plattformübergreifend und kompatibel mit Betriebssystemen wie Windows, Linux und macOS. Die IDE basiert auf der Java-Plattform und unterstützt verschiedene Arduino-Module sowie die Programmierung in C und C++.

Die Mikrocontroller auf den Arduino-Boards sind so programmiert, dass sie die in Form von Code bereitgestellten Informationen verarbeiten. Programme, die in der IDE geschrieben werden, werden als Sketches bezeichnet. Diese Skizzen erzeugen eine Hex-Datei, die dann an den Mikrocontroller übertragen und hochgeladen wird.

Die IDE-Umgebung besteht aus zwei Hauptkomponenten: dem Editor und dem Compiler. Der Editor wird zum Schreiben von Code verwendet, während der Compiler den Code kompiliert und in das Arduino-Modul hochlädt.[FAD18] In der Version 2.3.3 der Arduino-IDE spielt die Menüleiste, die sich am oberen Rand der Benutzeroberfläche befindet, eine entscheidende Rolle, da sie Zugriff auf wichtige Befehle wie Hochladen, Überprüfen/Kompilieren und Speichern bietet. Darüber hinaus enthält sie das Dateimenü zum Öffnen neuer oder vorhandener Dateien und den Abschnitt Beispiele, der vorgefertigte Skizzen für verschiedene Anwendungen wie Blink und Fade bietet.

Die 6 Schaltflächen am oberen Rand des Bildschirms sind wie folgt:

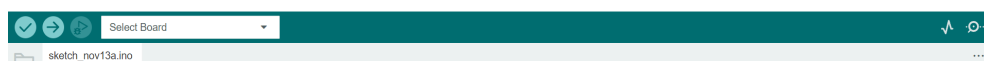


Abbildung 1.1.: Menu Button

-  Das Häkchen im Symbol wird verwendet, um den geschriebenen Code zu überprüfen. Nachdem der Code eingegeben wurde, wird durch die Auswahl dieses Symbols auf Fehler geprüft und bestätigt, dass der Code richtig strukturiert ist. Dieser Schritt stellt sicher, dass der Code für die Verwendung mit der Hardware bereit ist.
-  Das Symbol **Upload** in der Arduino IDE kompiliert Ihren Code und lädt ihn auf das angeschlossene Board hoch, so dass er lauffähig ist.
-  Das Symbol **Start Debugging** in der Arduino IDE startet eine Debugging-Sitzung, die es Ihnen ermöglicht, Ihren Code zu analysieren, indem Sie Haltepunkte setzen, durch die Ausführung schreiten und Variablen auf kompatiblen Boards untersuchen.

Das Icon **Serial Plotter** in der Arduino IDE öffnet ein Tool, das die vom Board über die serielle Verbindung gesendeten Echtzeitdaten visualisiert und zur einfacheren Analyse als Graphen anzeigt.

Das Symbol **Serial Monitor** in der Arduino IDE öffnet ein Terminal zum Anzeigen und Senden von textbasierten Daten über die serielle Verbindung und ermöglicht die Kommunikation mit dem angeschlossenen Board in Echtzeit.

VS: hier noch einmal die letzten Grafiken in der Auflistung nach links verschieben

## 1.2. Installation der Arduino IDE

### 1.2.1. Download der Arduino IDE

Der Arduino Nano 33 BLE Sense nutzt zur Programmierung die Arduino Software **Integrated Development Environment (IDE)**, die am weitesten verbreitete IDE für alle Arduino-Boards und kann sowohl online als auch offline ausgeführt werden. Diese Open-Source-IDE für Arduino vereinfacht das Schreiben von Code und das Hochladen auf das Board. Verschiedene Versionen der Software sind für jedes Betriebssystem (OS) verfügbar, einschließlich macOS, Linux und Windows. Die Arduino-Community bietet auch eine Online-Plattform für das Programmieren und Speichern von Skizzen in der Cloud. Dieser Online-Arduino-Editor ist die neueste Version der IDE und enthält alle Bibliotheken und Unterstützung für neue Arduino-Boards. Um auf diese Softwarepakete zuzugreifen, besuchen Sie die folgende Website: [Arduino-Software](#), um auf dem Laufenden zu bleiben, da unter dem genannten Link täglich Updates verfügbar sind.

Der Editor kann problemlos direkt von der Arduino-Software-Seite heruntergeladen werden. Die Download-Optionen sind in Abbildung 1.2 zu sehen.

## Downloads



### Arduino IDE 2.3.3

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

**SOURCE CODE**

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

#### DOWNLOAD OPTIONS

**Windows** Win 10 and newer, 64 bits

**Windows** MSI Installer

**Windows** ZIP file

**Linux** ApplImage 64 bits (X86-64)

**Linux** ZIP file 64 bits (X86-64)

**macOS** Intel, 10.15: "Catalina" or newer, 64 bits

**macOS** Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Abbildung 1.2.: Arduino IDE 2.3.3 Download

### 1.2.2. Zusammenfassung der Optionen

Nachfolgend finden Sie eine Zusammenfassung 1.3 der verfügbaren Download-Optionen für die Arduino IDE, die auf verschiedene Betriebssysteme und individuelle Bedürfnisse

zugeschnitten sind. Wählen Sie die passende Version für Ihr Gerät, um Kompatibilität und Zugang zu den neuesten Funktionen zu gewährleisten.

#### Summary of Options:

| Operating System | Option                                     | Description                                                                                |
|------------------|--------------------------------------------|--------------------------------------------------------------------------------------------|
| Windows          | Windows 10 and newer (64-Bit)              | Direct download for Windows 10 and newer 64-bit versions, without using the MSI installer. |
|                  | MSI Installer                              | Easy installation through an <code>.msi</code> package.                                    |
|                  | ZIP File                                   | Portable, no installation required, just extract and run directly.                         |
| Linux            | ApplImage 64-Bit (x86-64)                  | Portable, no installation required.                                                        |
|                  | ZIP File 64-Bit (x86-64)                   | Portable, extract and run directly.                                                        |
| macOS            | macOS Intel (10.15: Catalina or newer)     | For Intel Macs with macOS 10.15 or newer.                                                  |
|                  | macOS Apple Silicon (11: Big Sur or newer) | For Apple Silicon Macs (M1, M2) with macOS Big Sur 11 or newer.                            |

Abbildung 1.3.: Zusammenfassung der Download-Optionen

### 1.2.3. Installation der Arduino IDE für Windows

Der Download wird für die gewünschte Software ausgewählt Abbildung 1.4, in diesem Fall „Arduino IDE 2.3.2 für Windows10 and newer, 64 bits“. Nach Öffnen der Download Datei öffnet sich wie Abbildung 1.5 ein Fenster zur Abfrage des Lizenzabkommens.

## Downloads



## Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

#### DOWNLOAD OPTIONS

**Windows** Win 10 and newer, 64 bits

**Windows** MSI installer

**Windows** ZIP file

**Linux** ApplImage 64 bits (X86-64)

**Linux** ZIP file 64 bits (X86-64)

**macOS** Intel, 10.15: "Catalina" or newer, 64 bits

**macOS** Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Abbildung 1.4.: Downloadfenster Arduino IDE

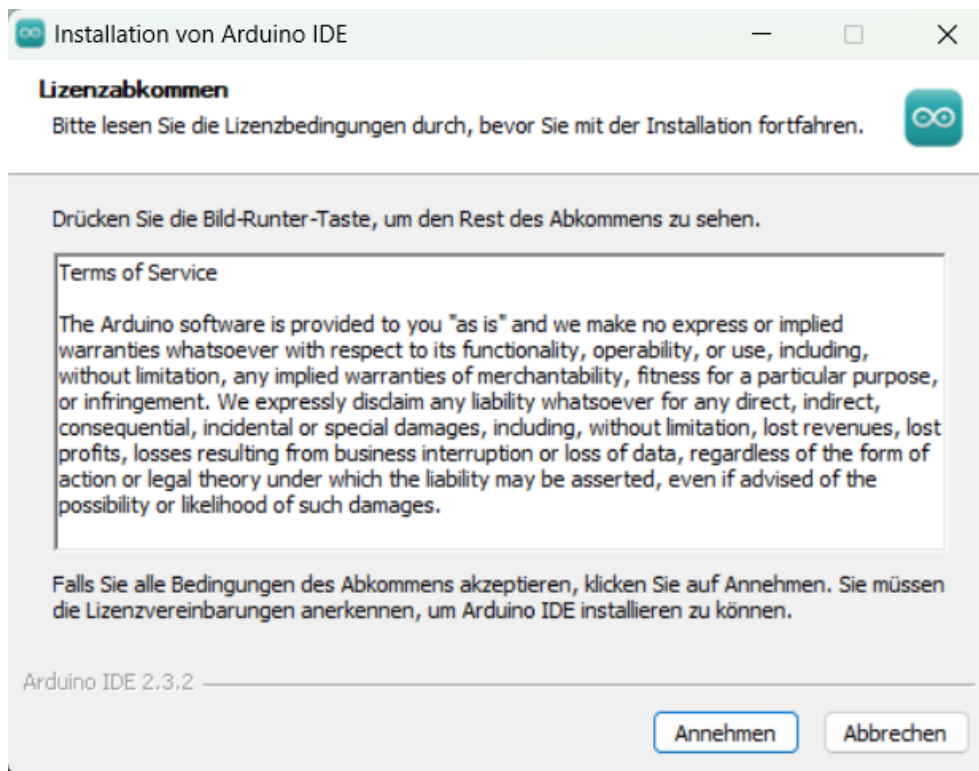


Abbildung 1.5.: Lizenzabkommen

Die 1.6 zeigt das darauf folgende Fenster zur Abfrage auf welchen Konto auf dem PC das Programm installiert werden soll.

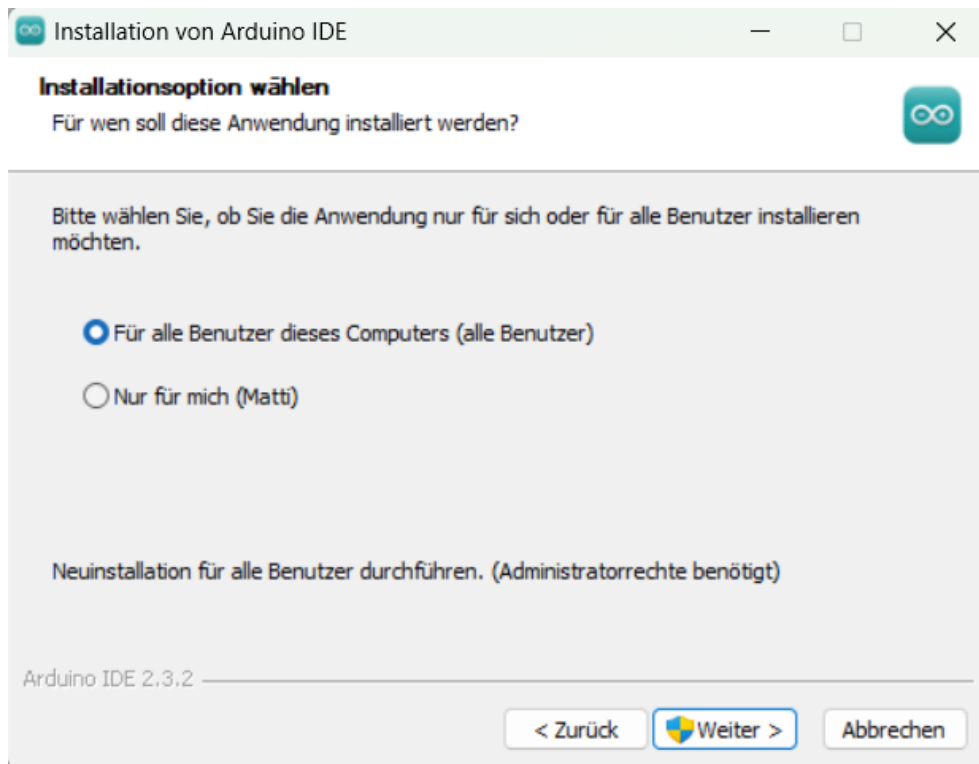


Abbildung 1.6.: Installationsoption

Die Installation FFür alle Benutzer dieses Computers"benötigt der Installer die Administrator rechte, nach der Genehmigung muss erneut das Lizenzabkommen bestätigt werden Abbildung 1.5. Darauf folgt Abbildung 1.7, die Auswahl des Zielverzeichnisses auf dem PC.

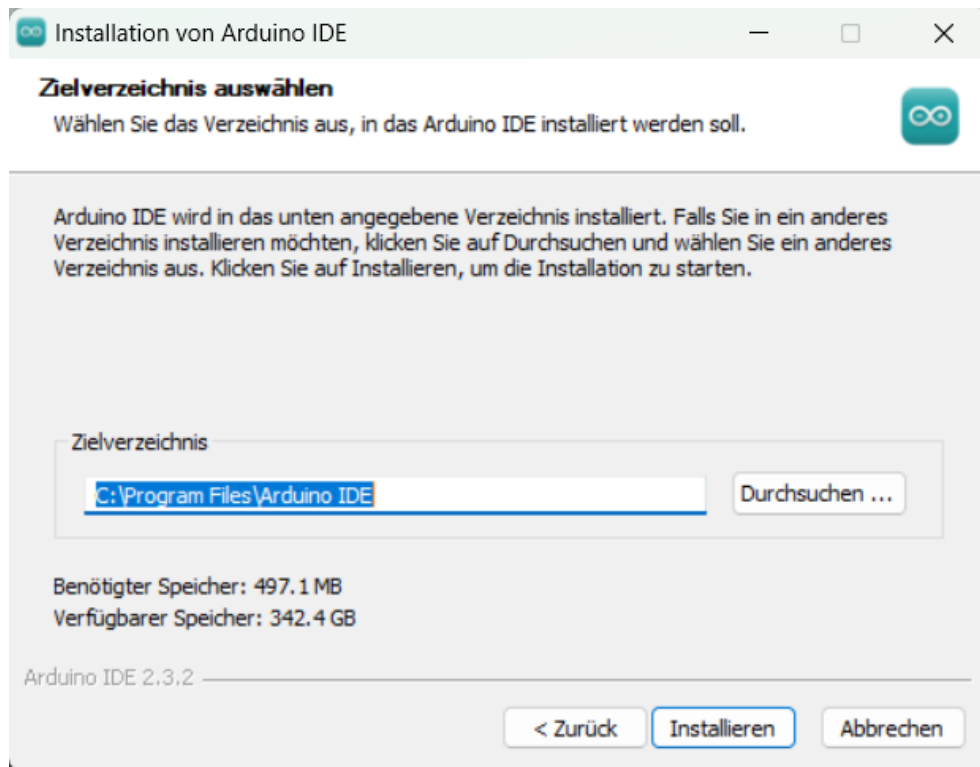


Abbildung 1.7.: Zielverzeichnisauswahl

Nach der Installation wird diese durch Fertigstellen beendet. Nun kann die Arduino IDE geöffnet werden.

### 1.3. Verknüpfen des Boards mit der IDE

Zur Verbindung des Arduino Boards mit der Arduino IDE muss in dieser zuerst das richtige Board ausgewählt werden Abbildung 1.8. Dazu wird in der Menüleiste unter dem Punkt **Tools** der Unterpunkt **Board** ausgewählt. Unter den Boards „Arduino AVR“ und hier der „Arduino Nano“ ausgewählt.

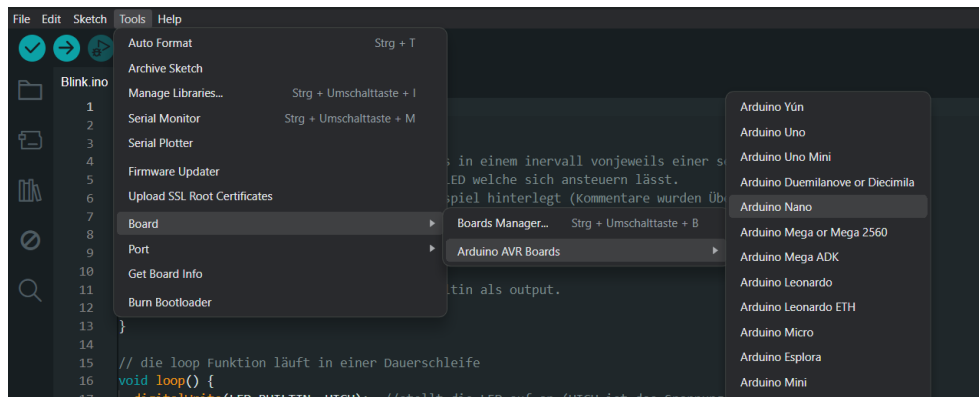


Abbildung 1.8.: Boardauswahl

Nach der Boardauswahl wird nun wieder in der Menüleiste unter dem Punkt **Tools** der Unterpunkt **Ports** ausgewählt Abbildung 1.9. Hierunter befinden sich die Anschlüsse des PC. Hier kann es sein, dass hier bereits erkennbar ist, an welchem Port der Arduino angeschlossen ist. Ist dies nicht der Fall, trennt man den Arduino vom PC, der nun nicht mehr angezeigte Port ist der richtige. Der Arduino kann nun wieder angeschlossen werden und der Port ausgewählt werden.

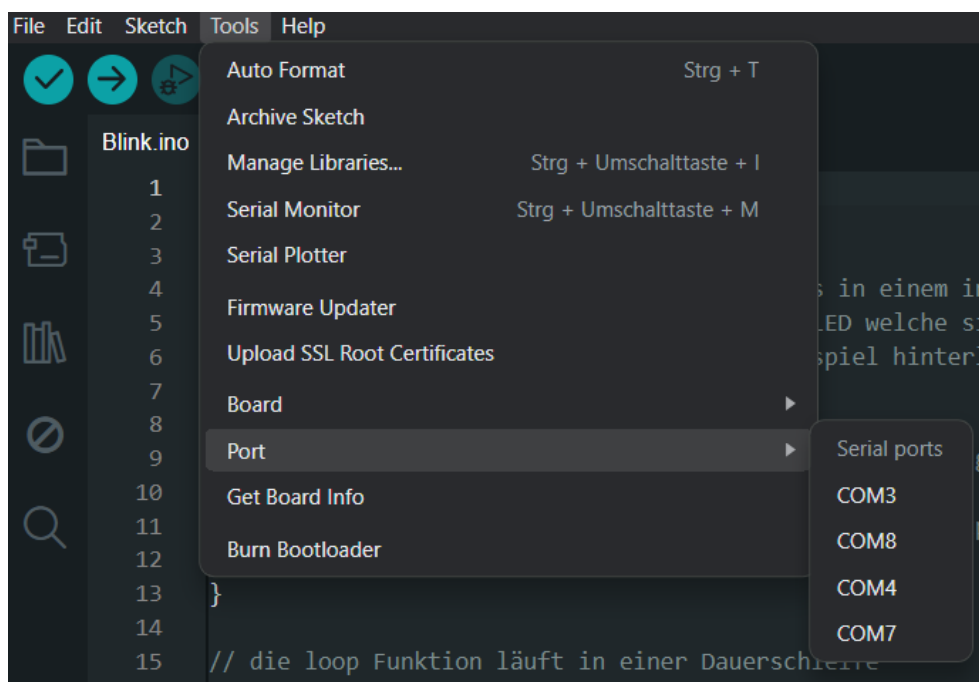


Abbildung 1.9.: Portauswahl

### 1.3.1. Beispielcode „Blinkende LED“

Der Aufbau und die Funktion der Programmierung in der Arduino IDE wird anhand des Beispiels Blinkenden LED gezeigt und erklärt Abbildung 1.1. Die LED ist auf dem Arduino Board fest installiert, somit benötigt das Beispiel keine Externe Hardware.

Listing 1.1.: Blink Programm



```

1000 /**
1001  * @file blink.ino
1002  * @brief Simple example to blink the on-board LED on and off.
1003  *
1004  * @details
1005  * Turns the built-in LED on for one second, then off for one second
1006  * repeatedly.
1007  * Most Arduino boards have a built-in LED connected to a specific
1008  * digital pin.
1009  * The macro LED_BUILTIN is used to ensure compatibility across
1010  * different boards.
1011  *
1012  * @note For more details about your board's built-in LED, see:
1013  *       https://www.arduino.cc/en/Main/Products
1014  *
1015  * @see https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
1016  *
1017  * @author
1018  * - Scott Fitzgerald (original)
1019  * - Arturo Guadalupi (modification)
1020  * - Colby Newman (modification)
1021  *
1022  * @date Modified:
1023  * - 8 May 2014
1024  * - 2 Sep 2016
1025  * - 8 Sep 2016
1026  *
1027  * @copyright Public Domain
1028  */
1029
1030 /**
1031  * @brief Initializes the LED pin as an output.
1032  *
1033  * This function runs once at startup after reset or power-up.
1034  */
1035 void setup() {
1036   pinMode(LED_BUILTIN, OUTPUT); ///< Set the LED pin as output
1037 }
1038
1039 /**
1040  * @brief Toggles the LED on and off every second.
1041  *
1042  * This function runs repeatedly after setup().
1043  */
1044 void loop() {
1045   digitalWrite(LED_BUILTIN, HIGH); ///< Turn the LED on
1046   delay(1000);                    ///< Wait for one second
1047   digitalWrite(LED_BUILTIN, LOW);  ///< Turn the LED off
1048   delay(1000);                    ///< Wait for one second
1049 }

```

../Code/Blink/Blink.ino

In den Zeilen 32 bis 34 steht die **Setup Funktion**, diese setzt in Zeile 33 über **pinMode** den Ausgang auf die integrierte LED über **LED BUILTIN** und setzt diesen als Output. Das Blinken wird in Zeile 41 bis 46 über die **loop** Funktion realisiert. In Zeile 42 wird über **digitalWrite** wird die LED auf **HIGH** gesetzt und ist damit an. Zeile 43 sorgt mit der Funktion **delay(1000)** für eine Pause von einer Sekunde, in der der Zustand der LED gleich bleibt. In Zeile 44 wird nun wieder über **digitalwrite** der zustand der LED auf **LOW** gesetzt und die LED ist aus. Zeile 45 ist nun wieder eine Pause von einer Sekunde. Dannach wird die **loop** Funktion endlos wiederholt.

### 1.3.2. Installation der Arduino IDE für MacOS

Der Download der Arduino IDE erfolgt über die Website . Geben Sie dazu „Arduino IDE Download “in Ihren Browser ein (vgl. Abbildung 1.10).

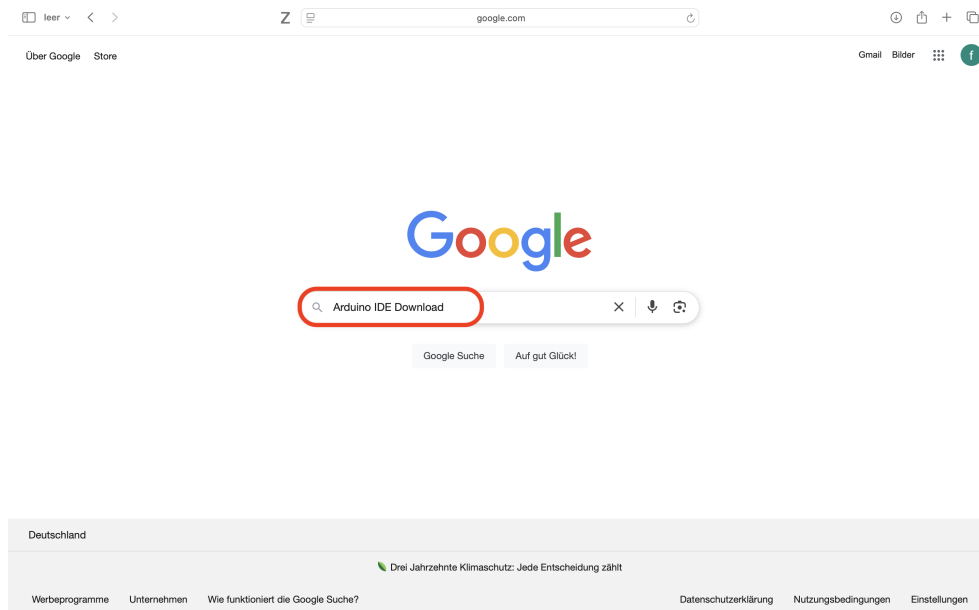


Abbildung 1.10.: Aufruf der Arduino-Webseite über die Suchmaschine

Abbildung 1.11 zeigt die Ergebnisse der Websuche. Klicken Sie auf den Eintrag mit „Arduino.cc“im Adresskopf.

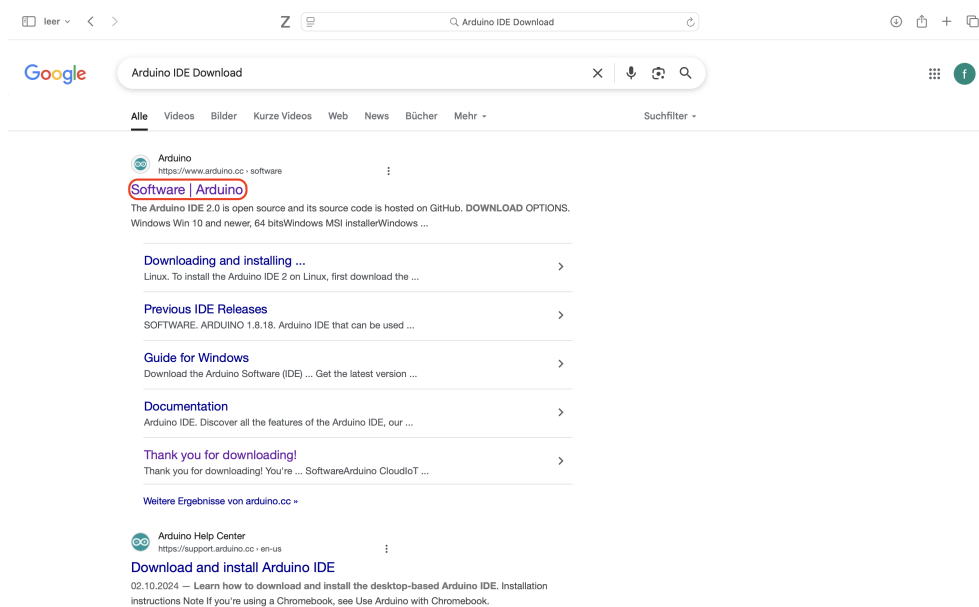


Abbildung 1.11.: Ergebnisse der Websuche zur Arduino IDE

Wählen Sie die passende Version für Ihren Mac aus (vgl. Abbildung 1.12).

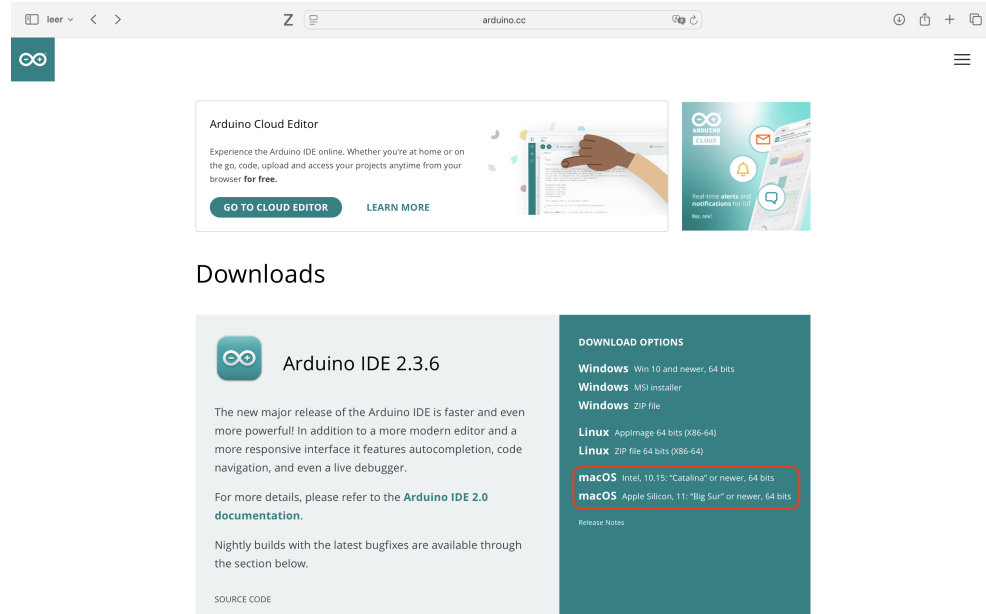


Abbildung 1.12.: Download der Arduino IDE starten

Es erscheint eine Spendenanfrage. Sie können freiwillig spenden oder über „JUST DOWNLOAD“ fortfahren (vgl. Abbildung 1.13).

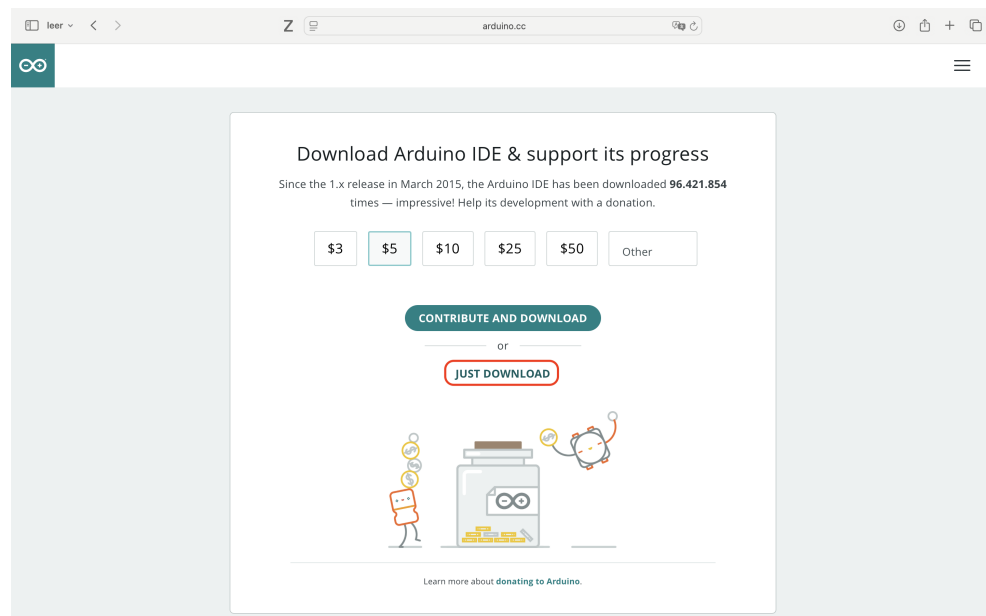


Abbildung 1.13.: Optional: Spendenaufruf beim Download

Vor dem Download können Sie optional den Newsletter abonnieren, vgl. Abbildung 1.14.

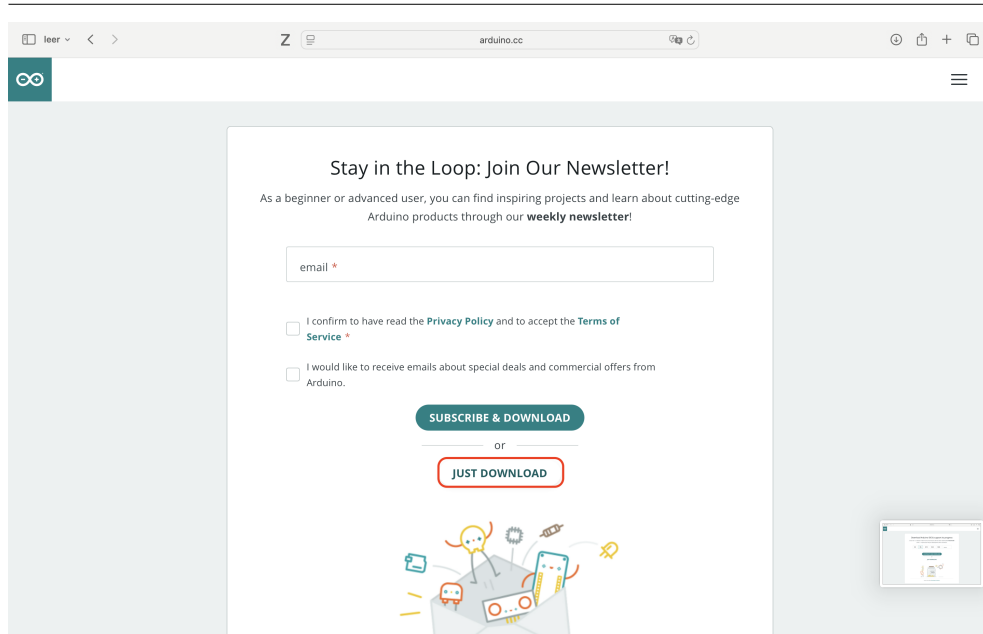


Abbildung 1.14.: Optional: Newsletter abonnieren

Da es sich um ein Programm aus dem Internet handelt, muss der Download zunächst erlaubt werden, vgl. Abbildung 1.15.

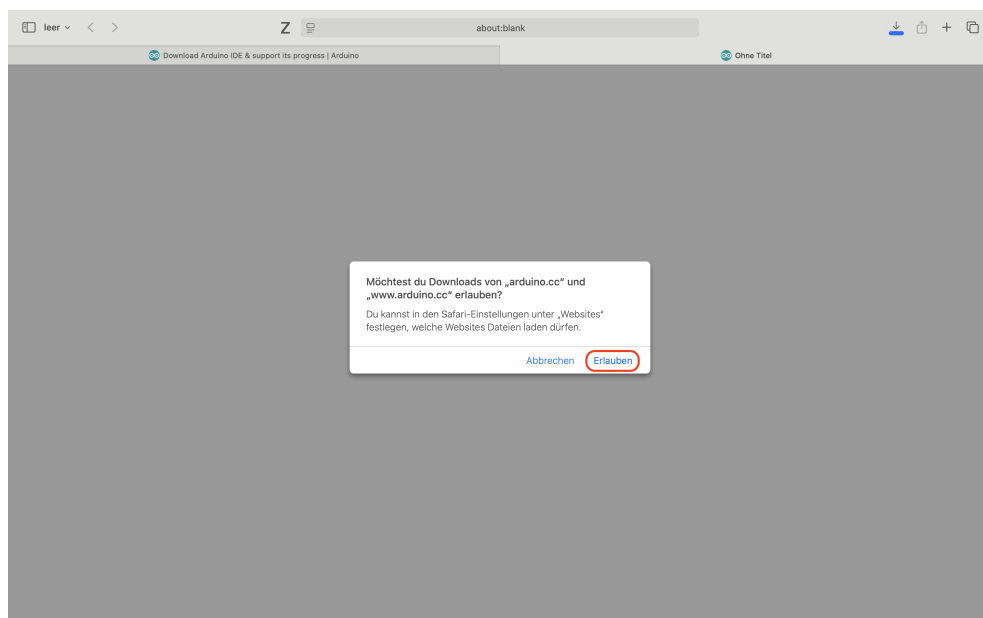


Abbildung 1.15.: Download-Erlaubnis erteilen

Über den Pfeil oben rechts lässt sich der Fortschritt anzeigen (vgl. Abbildung 1.16).

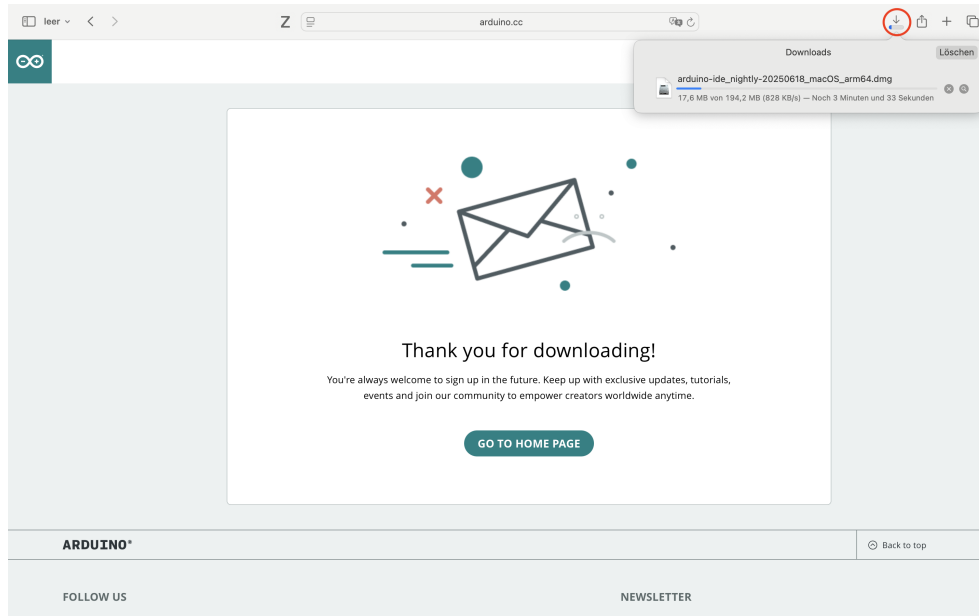


Abbildung 1.16.: Download-Fortschritt einsehen

Öffnen Sie nach dem Download die Datei per Doppelklick (vgl. Abbildung 1.17).

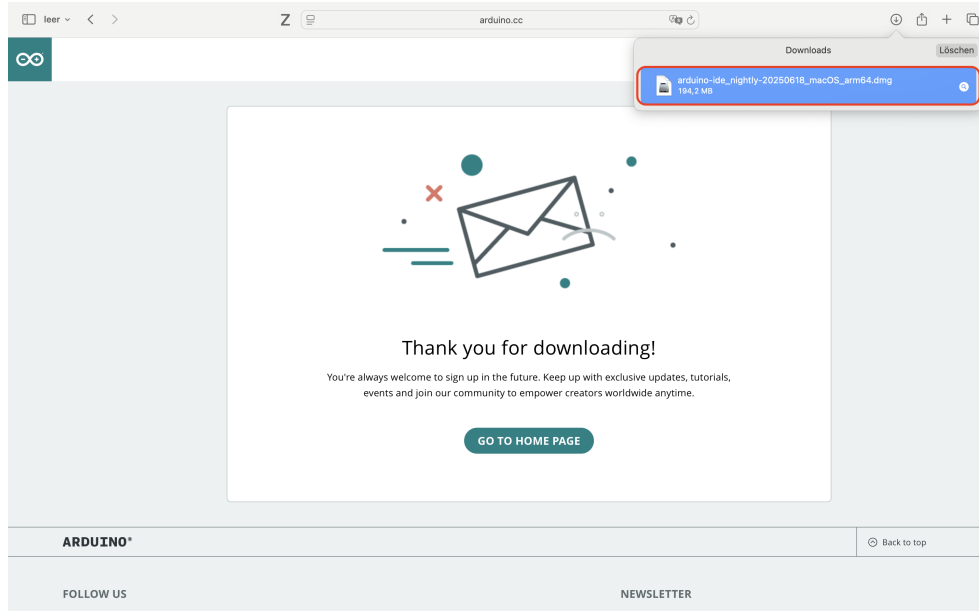


Abbildung 1.17.: Heruntergeladene Datei öffnen

Ziehen Sie nun die App in den Programme-Ordner (vgl. Abbildung 1.18).



Abbildung 1.18.: Installation durch Ziehen in den Programme-Ordner

Die Installation ist abgeschlossen. Öffnen Sie die App über den Programme-Ordner (vgl. Abbildung 1.19).

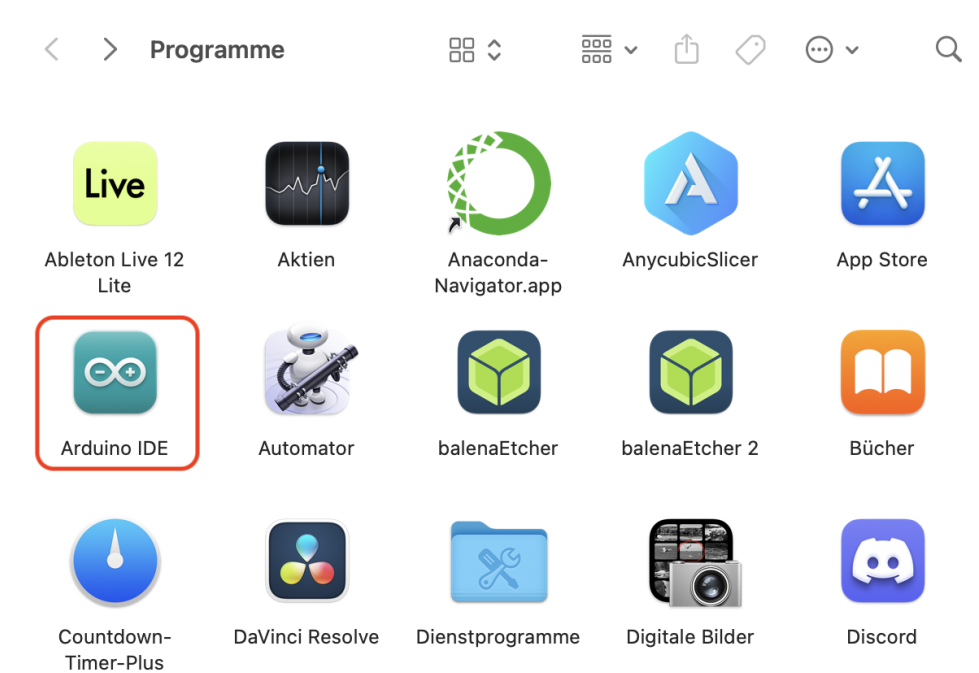


Abbildung 1.19.: Arduino IDE starten

Bestätigen Sie die Sicherheitsabfrage mit „Öffnen“ (vgl. Abbildung 1.20).



Abbildung 1.20.: Sicherheitsabfrage bei erstmaligem Start

### 1.3.3. Bibliotheken installieren

Klicken Sie auf das Buch-Symbol (vgl. Abbildung 1.21) und suchen Sie die gewünschte Bibliothek. Die vollständige Liste finden Sie in Kapitel ???. Installieren Sie jede Bibliothek durch Klick auf „Install“.

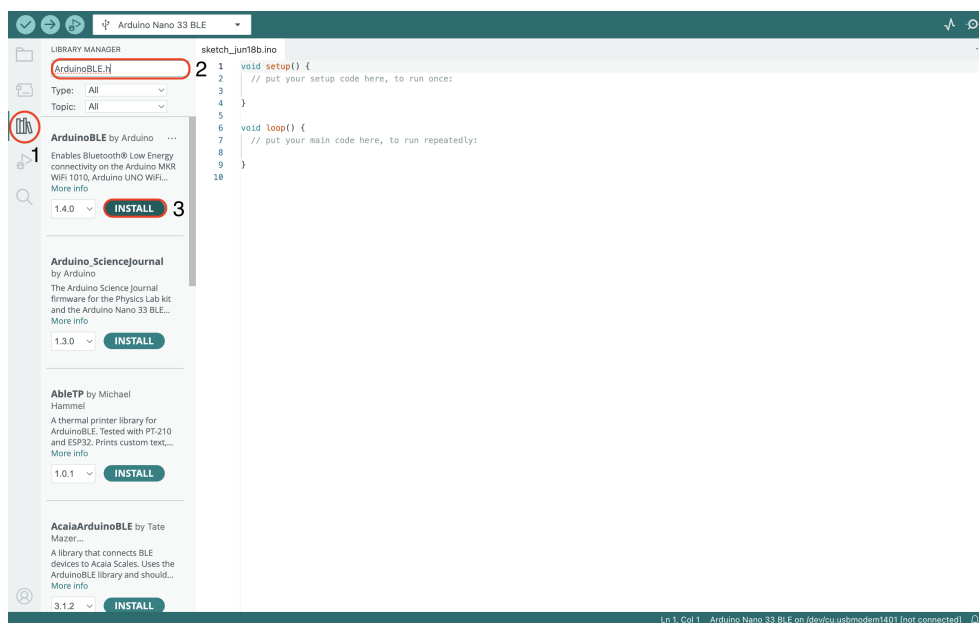


Abbildung 1.21.: Bibliotheken-Suchmaske der Arduino IDE

Abbildung 1.22 zeigt eine erfolgreich installierte Bibliothek.

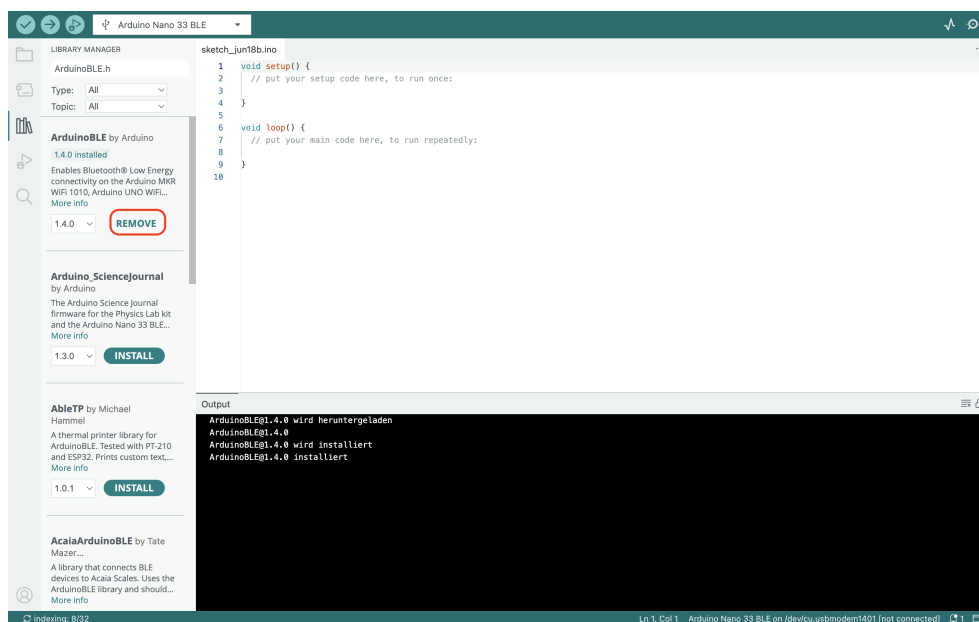


Abbildung 1.22.: Erfolgreiche Installation einer Bibliothek

### 1.3.4. Board Manager

Mit dem Board Manager können Sie zusätzliche Boards hinzufügen (vgl. Abbildung 1.23). Suchen Sie zum Beispiel nach dem „ESP“ oder „Nano 33 BLE Sense“ und installieren Sie das passende Board.



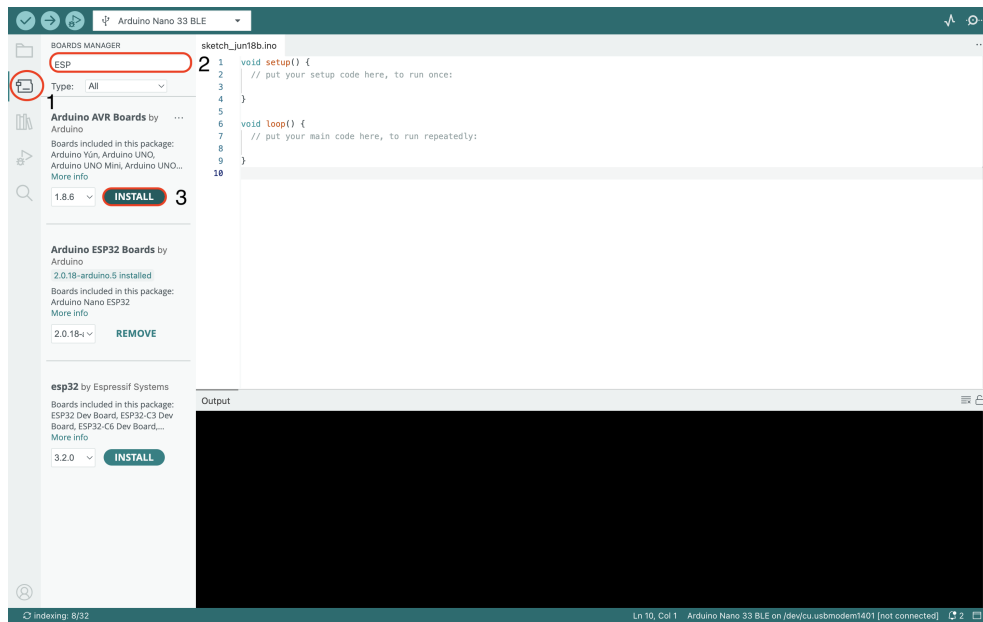


Abbildung 1.23.: Der Board Manager in der Arduino IDE

Klappen Sie das USB-Menü aus (vgl. Abbildung 1.24) und wählen Sie „Select other board and port“.

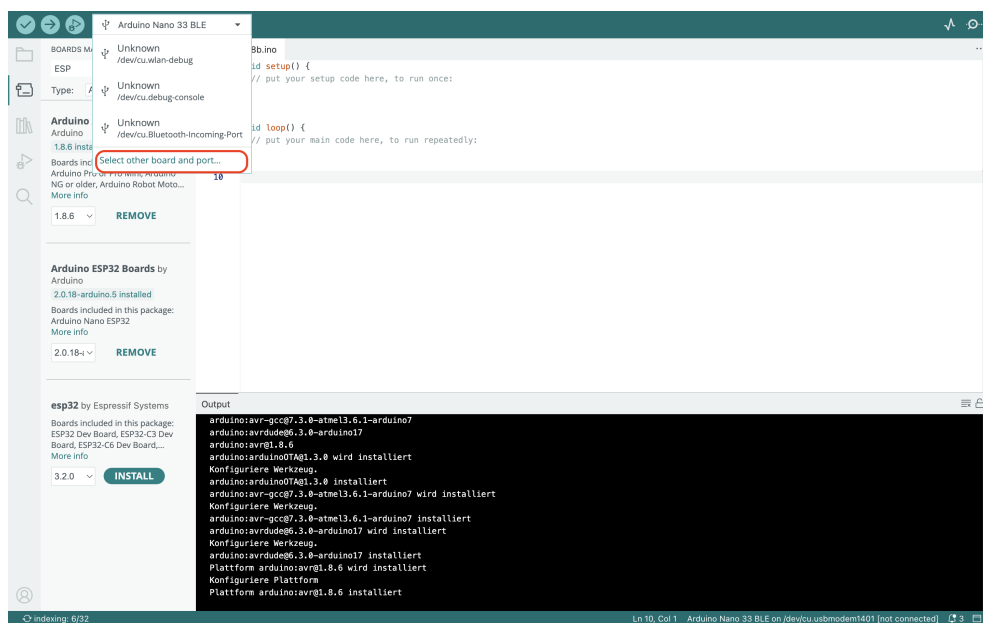


Abbildung 1.24.: Board- und Portauswahl öffnen

Geben Sie „Nano 33 BLE Sense“ in das Suchfeld ein, verbinden Sie das Board und wählen Sie den richtigen Port (vgl. Abbildung 1.25).

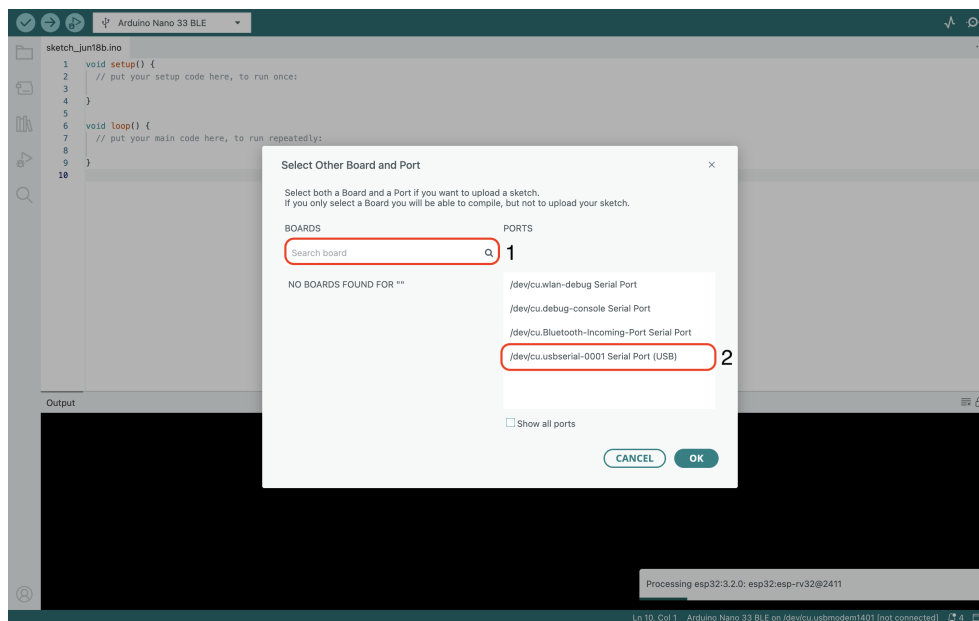


Abbildung 1.25.: Passenden Board und Port auswählen

### 1.3.5. Beispielprogramm „Blink“

Wählen Sie den Pfad gemäß Abbildung 1.26, um ein Beispielprogramm zu laden.

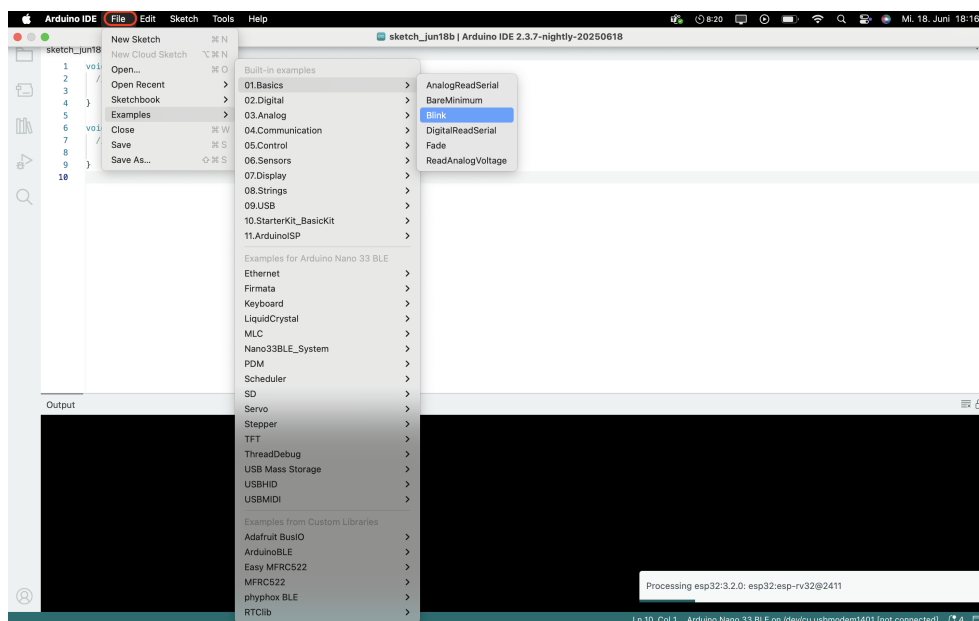
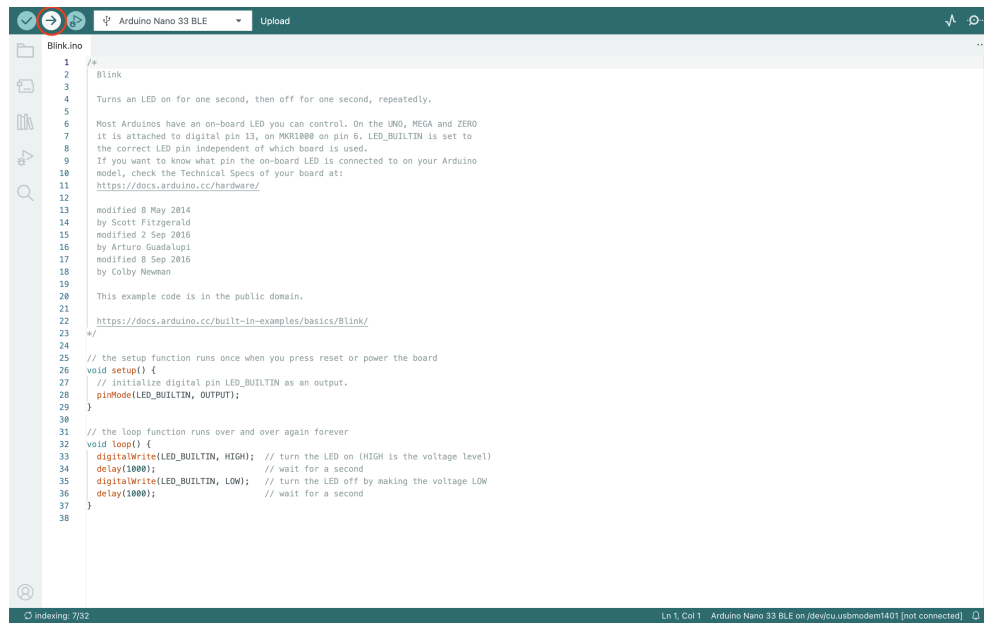


Abbildung 1.26.: Beispielprogramme öffnen

Das Blink-Programm wird geöffnet (vgl. Abbildung 1.27) und kann nach Auswahl des Ports hochgeladen werden (siehe 1.3.4). Die Kompilierung erfolgt automatisch vor dem Upload oder manuell über das Häkchen-Symbol.

The image shows the Arduino IDE interface with the 'Blink' example code loaded. The top toolbar includes icons for opening files, saving, and uploading. The menu bar shows 'File', 'Edit', 'Tools', and 'Help'. The 'Tools' menu is open, showing 'Arduino Nano 33 BLE' selected. The code editor displays the Blink.ino file, which contains a series of comments explaining the code and a C++ program that toggles the built-in LED on and off every second. The status bar at the bottom indicates 'Ln 1, Col 1' and 'Arduino Nano 33 BLE on /dev/cu.usbmodem1401 [not connected]'.

```
1  /*  
2  Blink  
3  
4  Turns an LED on for one second, then off for one second, repeatedly.  
5  
6  Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO  
7  it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to  
8  the correct LED pin independent of which board is used.  
9  If you want to know what pin the on-board LED is connected to on your Arduino  
10 model, check the Technical Specs of your board at:  
11 https://docs.arduino.cc/hardware/  
12  
13 modified 8 May 2014  
14 by Scott Fitzgerald  
15 modified 2 Sep 2016  
16 by Arturo Guadalupi  
17 modified 8 Sep 2016  
18 by Colby Newman  
19  
20 This example code is in the public domain.  
21  
22 https://docs.arduino.cc/built-in-examples/basics/Blink/  
23 */  
24  
25 // the setup function runs once when you press reset or power the board  
26 void setup() {  
27   // initialize digital pin LED_BUILTIN as an output.  
28   pinMode(LED_BUILTIN, OUTPUT);  
29 }  
30  
31 // the loop function runs over and over again forever  
32 void loop() {  
33   digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)  
34   delay(1000); // wait for a second  
35   digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW  
36   delay(1000); // wait for a second  
37 }  
38
```

Abbildung 1.27.: Das geöffnete Beispielprogramm „Blink“

Nach erfolgreichem Upload erscheint ein Terminalfenster mit einer Bestätigung.



## 2. Arduino IDE 2.3.6

Dieses Kapitel liefert eine detaillierte Einführung in die Arduino IDE, einschließlich ihrer zentralen Funktionen, verfügbaren Schnittstellen und grundlegenden Arbeitsweise. Es erläutert die einzelnen Bestandteile der Entwicklungsumgebung, beschreibt den Installationsprozess anhand klar strukturierter Schritte und bietet eine Anleitung zur Ersteinrichtung sowie zur grundlegenden Konfiguration der Programmierungsumgebung.

### 2.1. Arduino IDE Beschreibung

Die Arduino IDE ist die offizielle, Open-Source Entwicklungsumgebung zur Programmierung und Verwaltung von Arduino Mikrocontrollern. Sie ermöglicht das Schreiben, Kompilieren und Übertragen von Quellcode auf eine Vielzahl unterstützter Boards. Als plattformübergreifende Anwendung ist sie mit allen gängigen Betriebssystemen wie Windows, Linux und macOS kompatibel. Ursprünglich auf der Java-Plattform basierend, bietet die IDE Unterstützung für zahlreiche Arduino-Boards sowie eine auf C und C++ basierende Programmiersprache. Durch ihre intuitive Benutzeroberfläche richtet sie sich sowohl an Einsteiger als auch an erfahrene Entwickler im Bereich der Mikrocontroller-Programmierung [FAD18].

Die Mikrocontroller auf Arduino-Boards werden mithilfe von Programmen gesteuert, die in Form von Quellcode verfasst und über die Entwicklungsumgebung verarbeitet werden. In der Arduino IDE erstellte Programme werden als sogenannte „Sketche“ bezeichnet. Dabei handelt es sich um Skripte, die den Funktionsablauf des Mikrocontrollers definieren – etwa das Einlesen von Sensorwerten, die Ansteuerung von Aktoren oder die Kommunikation mit anderen Geräten. Nach dem Schreiben wird der Sketch von der IDE kompiliert und in eine maschinenlesbare Hex-Datei übersetzt. Diese Datei wird im Anschluss über die serielle Schnittstelle auf den Mikrocontroller hochgeladen, wo sie dauerhaft gespeichert und beim nächsten Start ausgeführt wird [FAD18].

Die Arduino-IDE setzt sich im Wesentlichen aus zwei funktionalen Kernkomponenten zusammen: dem Editor und dem Compiler. Der integrierte Editor dient als zentrale Arbeitsfläche zum Schreiben und Bearbeiten von Quellcode, während der Compiler für die Übersetzung des Codes in maschinenlesbare Anweisungen sowie dessen Übertragung auf das angeschlossene Arduino-Modul verantwortlich ist [FAD18].

In der Version 2.3.6 der Arduino IDE spielt die Menüleiste am oberen Rand der Benutzeroberfläche eine zentrale Rolle. Sie bietet direkten Zugriff auf essentielle Funktionen wie das Hochladen von Sketchen auf das Board, das Kompilieren zur Überprüfung auf Fehler sowie das Speichern von Projekten. Unter dem Menüpunkt „Datei“ lassen sich neue Sketche anlegen, bestehende Projekte öffnen oder speichern. Ein weiterer bedeutender Abschnitt ist „Beispiele“ – dort finden sich eine Vielzahl vorgefertigter Sketches, die typische Anwendungsszenarien demonstrieren, darunter klassische Programme wie „Blink“ zur LED-Steuerung oder „Fade“ zur Helligkeitssteuerung mittels Pulsweitenmodulation. Diese Beispiele erleichtern insbesondere Einsteigern den Zugang zur Mikrocontroller-Programmierung und dienen oft als Ausgangspunkt für eigene Entwicklungen [FAD18].

Die fünf Schaltflächen am oberen Bildschirmrand sind wie folgt:



-  Das Häkchen-Symbol wird verwendet, um den geschriebenen Code zu **Überprüfen**, nachdem der Code eingegeben wurde, wird der Code auf Syntaxfehler geprüft. Dieser Schritt stellt sicher, dass der Code für die Verwendung mit der Hardware bereit ist.
-  Das Symbol **Hochladen** in der Arduino IDE kompiliert den Code und lädt ihn auf den angeschlossenen Arduino hoch.
-  Das Symbol **Start Debugging** in der Arduino IDE startet eine Debugging Sitzung. Damit kann der Code analysiert werden, indem Haltepunkte gesetzt werden. Beim Debuggen wird die Ausführung schrittweise durchlaufen und Variablen auf Kompatibilität und Funktion geprüft, ohne das ganze Programm in einem Stück zu starten.
-  Das Symbol **Serial Plotter** in der Arduino IDE öffnet ein Tool, das Echtzeitdaten, die vom Board über die serielle Verbindung gesendet werden, grafisch darstellt. Dadurch wird die Analyse der Daten erleichtert.
-  Das Symbol **Serial Monitor** in der Arduino IDE öffnet ein Terminal, um textbasierte Daten über die serielle Verbindung anzuzeigen und zu senden. Dadurch wird eine Echtzeitkommunikation mit dem angeschlossenen Board ermöglicht.

## 2.2. Installation der Arduino IDE

### 2.2.1. Download oder Arduino IDE

Der Arduino Nano 33 BLE Sense wird über die Arduino Integrated Development Environment (IDE) programmiert, die als Standardumgebung für sämtliche Arduino-Boards etabliert ist. Sie zählt zu den am weitesten verbreiteten Entwicklungsplattformen im Bereich der Mikrocontroller-Programmierung. Die IDE ist sowohl als Offline-Variante zur lokalen Installation als auch als Online-Editor verfügbar. Als Open-Source-Software ist sie darauf ausgelegt, den Programmierprozess möglichst zugänglich zu gestalten – vom Schreiben des Codes bis hin zum Hochladen auf das jeweilige Board.

Für alle gängigen Betriebssysteme – einschließlich macOS, Linux und Windows – stehen passende Versionen zur Verfügung. Die Installation ist unkompliziert und wird durch die breite Unterstützung der Arduino-Community zusätzlich erleichtert. Neben der Desktop-Version bietet Arduino auch eine cloudbasierte Entwicklungsumgebung an: den Online-Arduino-Editor. Diese moderne Webanwendung ermöglicht das plattformunabhängige Programmieren, Speichern und Verwalten von Sketchen direkt im Browser. Sie enthält alle gängigen Bibliotheken und bietet sofortige Unterstützung für aktuelle Arduino-Boards, einschließlich des Nano 33 BLE Sense.

Zugriff auf die verschiedenen Softwarepakete erhalten Sie über die offizielle Website der Arduino-Entwickler: [Arduino-Software](https://www.arduino.cc/en/software). Dort finden sich stets die neuesten Versionen der IDE sowie weiterführende Informationen. Eine Übersicht der verfügbaren Downloadoptionen ist in Abbildung 2.1 dargestellt.



Abbildung 2.1.: Arduino IDE 2.3.6

### 2.2.2. Anforderungen Betriebssystem

Im Folgenden sind die Systemanforderungen und Varianten der verfügbaren Downloadoptionen für die Arduino-IDE aufgeführt. Diese sind jeweils auf unterschiedliche Betriebssysteme und Nutzerbedürfnisse zugeschnitten. Um einen reibungslosen Betrieb sowie den Zugriff auf alle aktuellen Funktionen und Sicherheitsupdates zu gewährleisten, sollte stets die zur eigenen Plattform passende Version der IDE ausgewählt werden.

- Windows– Win 10 und neuer, 64 Bit
- Linux– 64 Bit
- macOS– Intel, Version 10.15: „Catalina“ oder neuer, 64 Bit
- macOS– Apple Silicon, 11: „Big Sur“ oder neuer, 64 Bit

### 2.2.3. Installation auf Windows

Die Installation der Arduino IDE auf einem Windows-System wird im folgenden Abschnitt Schritt für Schritt erläutert. Im Anschluss daran folgt eine Einführung in zentrale Funktionen der Umgebung, darunter der Board-Manager, der Bibliotheks-Manager, das Sketchbook sowie weitere essentielle Werkzeuge der Benutzeroberfläche.

Um die Arduino IDE auf einem Windows-Computer zu installieren, laden Sie zunächst die entsprechende Installationsdatei von der offiziellen Website unter [Arduino-Software](#) herunter. Starten Sie anschließend die heruntergeladene Datei und folgen Sie den angezeigten Installationsanweisungen. Diese beinhalten in der Regel die

Auswahl des Installationsverzeichnisses sowie optionaler Komponenten wie Verknüpfungen. Sobald der Vorgang abgeschlossen ist, steht die Entwicklungsumgebung zur sofortigen Nutzung bereit.

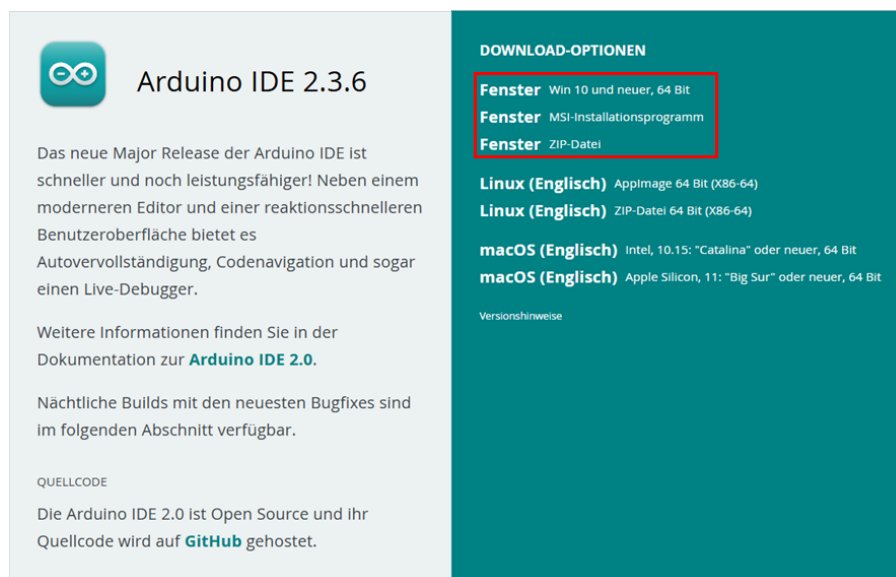


Abbildung 2.2.: Arduino IDE Downloadoptionen

Nachdem der Download der Arduino IDE abgeschlossen wurde, erscheint im nächsten Schritt das Fenster mit den Lizenzbedingungen. Diese sollten aufmerksam durchgelesen werden, bevor mit der Installation fortgefahren wird. Um die Installation zu starten, müssen alle Lizenzbedingungen akzeptiert werden. Dies geschieht durch einen Klick auf die Schaltfläche **Annehmen**, wie in Abbildung 2.3 dargestellt.

Im Anschluss wird der Benutzer aufgefordert, ein Zielverzeichnis für die Installation auszuwählen. Standardmäßig ist ein Pfad vorgegeben, dieser kann jedoch bei Bedarf individuell angepasst werden. Nachdem das gewünschte Verzeichnis ausgewählt wurde, wird der Installationsvorgang durch einen Klick auf **Installieren** gestartet, siehe Abbildung 2.4.



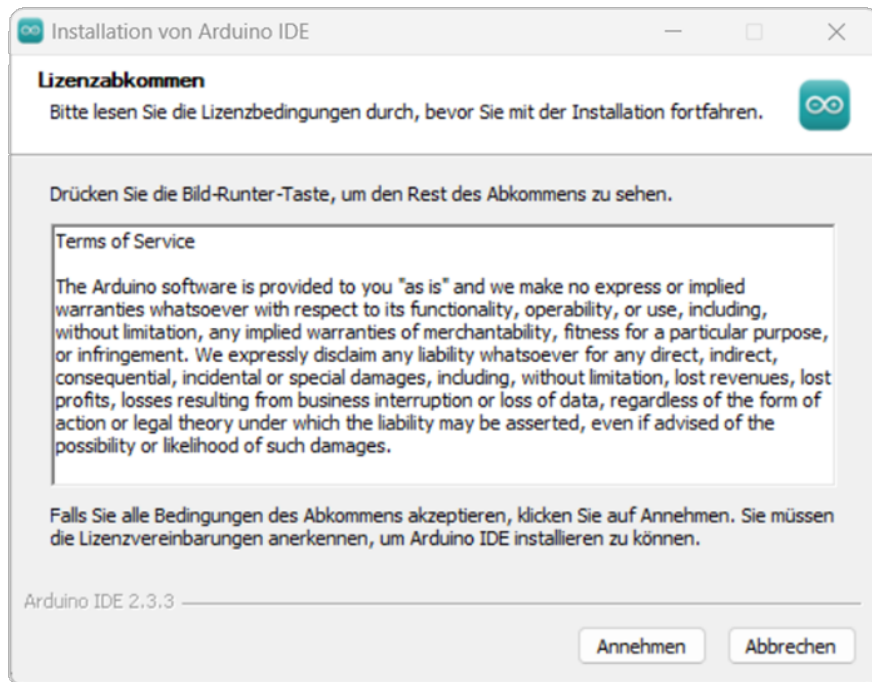


Abbildung 2.3.: Lizenzbestätigung Installation

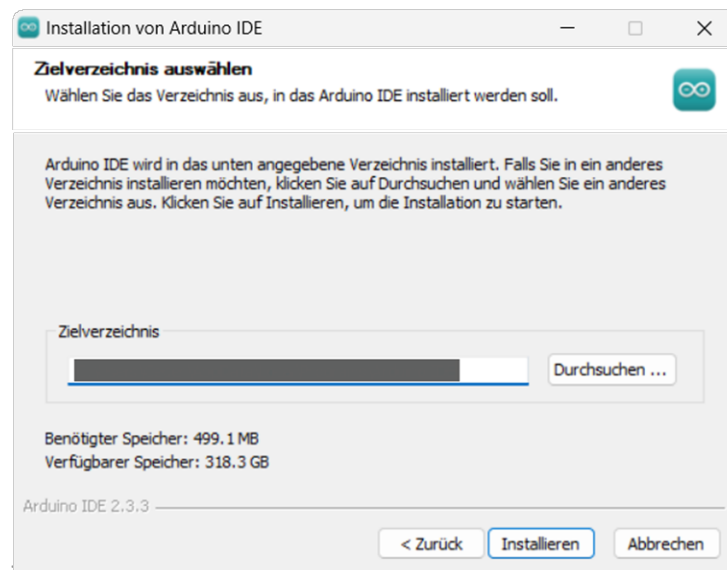


Abbildung 2.4.: Wahl Zielverzeichnis

Sobald die Installation der Arduino IDE erfolgreich abgeschlossen ist, wird die Anwendung automatisch gestartet. Beim ersten Öffnen der Arduino IDE wird ein Standard-Sketch angezeigt, der als Beispiel dient. Dieser ist im Startfenster sichtbar, wie in Abbildung 2.5 dargestellt.

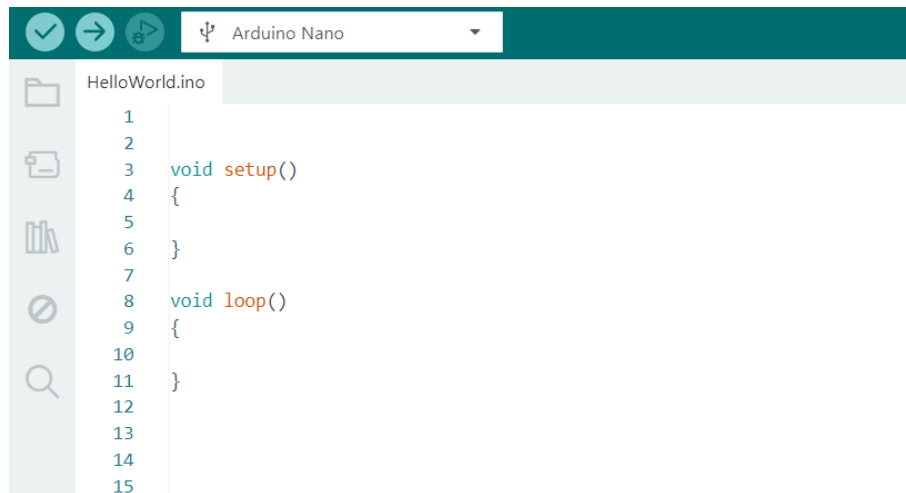


Abbildung 2.5.: Startfenster Arduino-IDE

Aus der oben dargestellten Abbildung 2.5 geht hervor, dass der grundlegende Arduino-Sketch aus zwei Hauptteilen besteht. Der erste Teil ist die Funktion `void setup()`, die keinen Rückgabewert liefert und in der typischerweise die Initialisierungen vorgenommen werden. Hier werden beispielsweise Einstellungen wie die Konfiguration der Ausgangs-LED, die Festlegung von Pin-Modi oder die Initialisierung von Variablen durchgeführt.

Der zweite Teil des Sketches ist die Funktion `void loop()`, in der der Hauptcode geschrieben wird. Funktionen, die wiederholt ausgeführt werden sollen, werden hier definiert und laufen fortlaufend in der Schleife. Diese Funktionen sind ebenfalls zwischen geschweiften Klammern `{ }` eingefasst. Jede Funktion in Arduino hat einen Rückgabewert – in diesem Fall handelt es sich um den Rückgabebetyp `void`, was bedeutet, dass die Funktion keinen Wert zurückgibt.

## 2.2.4. Installation (MacOS)

Um die Arduino IDE zu installieren, beginnen Sie mit dem Herunterladen der neuesten Version von der offiziellen Arduino-Website unter [Arduino-Software](#). Wählen Sie die Version, die mit Ihrem Betriebssystem kompatibel ist. In diesem Beispiel wird die Version Arduino 2.6.3 für macOS (Sonoma 14.4.1) verwendet. Die Setup-Datei trägt den Namen „Arduino-Software“ und hat eine Größe von etwa 193.600 KB. Diese Datei ist im Zip-Format.

Für Nutzer, die die Arduino IDE 2.3.6 mit Safari herunterladen, wird die Datei nach Abschluss des Downloads automatisch entpackt. Andere Browser hingegen erfordern möglicherweise eine manuelle Entpackung. Die folgende Abbildung 2.6 zeigt die neueste Offline-Version der Arduino IDE 2.3.6, die ebenfalls mit allen gängigen Betriebssystemen kompatibel ist.

Da in diesem Kapitel keine entsprechende Hardware für macOS zur Verfügung steht, wird der Installationsprozess für dieses Betriebssystem nicht im Detail behandelt.

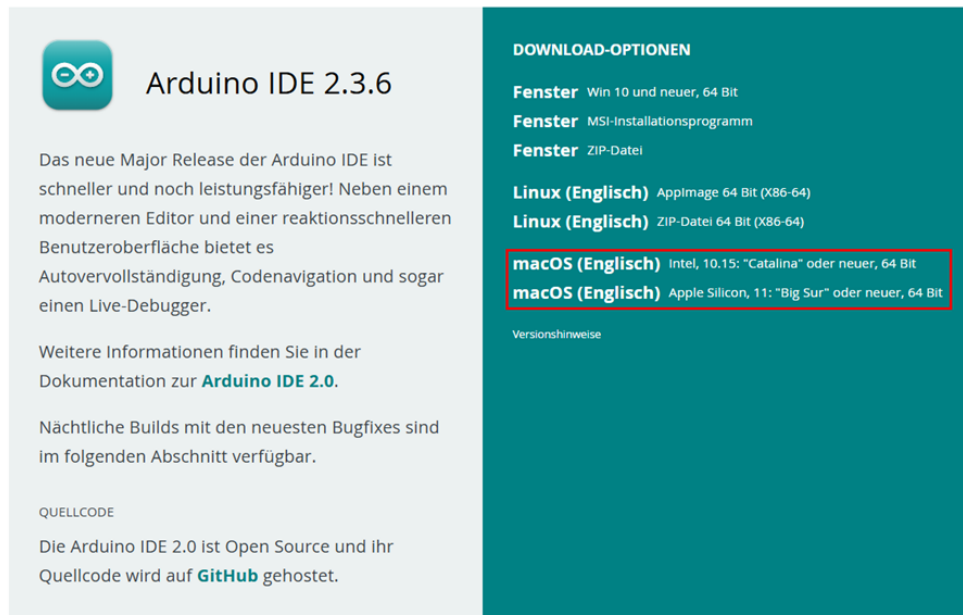


Abbildung 2.6.: ArduinoIDE macOS

## 2.3. Übersicht der Eigenschaften

Die Arduino IDE verfügt über eine neue Seitenleiste, die den Zugriff auf die am häufigsten verwendeten Werkzeuge erleichtert. 2.7

- **Verify / Upload** Diese Funktionen werden verwendet, um Code zu kompilieren und auf ein Arduino-Board hochzuladen.
- **Select Board and Port** Automatisch erkannte Arduino-Boards werden hier zusammen mit der Portnummer angezeigt.
- **Sketchbook** Dieser Bereich dient als zentrales Verzeichnis für alle Sketche, die lokal auf dem Computer des Benutzers gespeichert sind. Das Sketchbook bietet einen strukturierten, leicht zugänglichen Raum zum Verwalten, Organisieren und Bearbeiten von Code, der in der Arduino-Umgebung entwickelt wurde. Darüber hinaus bietet das Sketchbook eine Synchronisierungsfunktion mit der Arduino Cloud, die einen nahtlosen Zugriff auf gespeicherte Sketche über verschiedene Geräte hinweg ermöglicht. Diese Cloud-Integration erlaubt es Nutzern, Sketche aus der Online-Arduino-Umgebung abzurufen und zu bearbeiten.
- **Boards Manager** Durchsuchen Sie Arduino- und Drittanbieterpakete, die installiert werden können. Zum Beispiel erfordert die Verwendung eines MKR WiFi 1010-Boards die Installation des Arduino SAMD Boards-Pakets.
- **Library Manager** Durchsuchen Sie tausende Arduino-Bibliotheken, die von Arduino und seiner Community erstellt wurden.
- **Debugger** Programme in Echtzeit testen und debuggen.
- **Search** Nach Schlüsselwörtern im geschriebenen Code suchen.
- **Open Serial Monitor** Öffnet das Werkzeug „Serial Monitor“ als neuen Tab in der Konsole.



Abbildung 2.7.: Überblick

### 2.3.1. Sketchbook

Das Sketchbook ist der Ort, an dem Arduino-Code-Dateien - sogenannte Sketche - gespeichert werden. Diese Dateien haben die Dateiendung .ino. Um eine saubere Organisation zu gewährleisten, muss jeder Sketch in einem Ordner gespeichert werden, der genau denselben Namen wie die Sketch-Datei trägt. Beispiel: Ein Sketch namens MySketch.ino muss in einem Ordner namens MySketch gespeichert werden. Diese Namenskonvention ist notwendig, damit die Arduino IDE die Sketche korrekt erkennen und verwalten kann 2.8.

In der Regel werden Sketche in einem Ordner namens Arduino gespeichert, der sich im Dokumente-Verzeichnis des Systems befindet.

Um auf das Sketchbook zuzugreifen, kann in der Arduino IDE das Ordnersymbol in der Seitenleiste ausgewählt werden. Dadurch wird das Verzeichnis geöffnet, in dem die Sketche gespeichert sind, und ein effizienter Zugriff sowie eine einfache Verwaltung der Sketche ermöglicht.

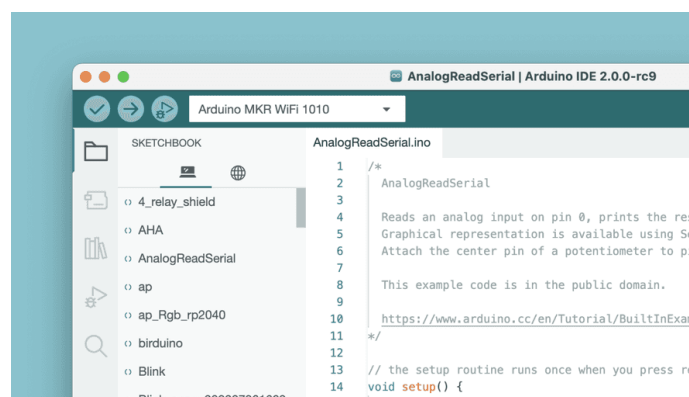


Abbildung 2.8.: Sketchbook

### 2.3.2. Boards Manager

Der Boards Manager kann über folgende Schritte in der Arduino IDE aufgerufen werden: **Tools** » **Board** » **Board Manager**.

Der Boards Manager ermöglicht die Installation verschiedener Board-Pakete, wie in Abbildung 2.9 dargestellt.

Ein Board-Paket enthält die notwendigen Dateien und Anweisungen, um Code für bestimmte Arduino-Boards zu kompilieren und hochzuladen.

Es gibt verschiedene Board-Pakete, z.B. avr, samd und megaavr. Jedes dieser Pakete unterstützt unterschiedliche Familien von Arduino-Boards:

- avr wird für ältere Boards wie den Arduino Uno verwendet,
- samd ist für neuere Boards wie den Arduino Zero gedacht.

Durch die Verwendung des Boards Managers wird sichergestellt, dass die richtigen Werkzeuge und Dateien für das gewählte Board installiert sind.

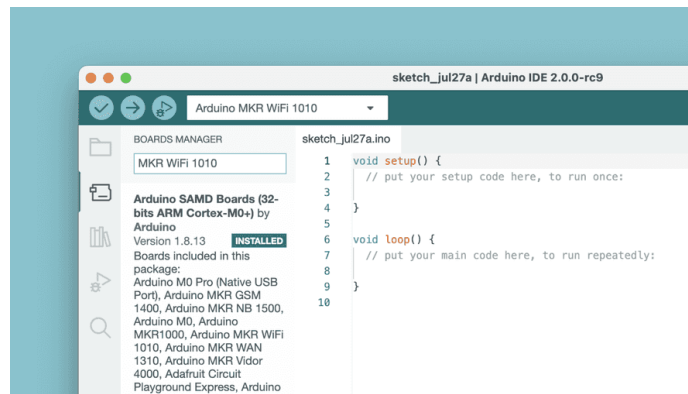


Abbildung 2.9.: Boards Manager

### 2.3.3. Library Manager

Der Library Manager kann in der Arduino IDE über folgende Schritte aufgerufen werden: **Tools** » **Manage Libraries**.

Der Bibliotheksverwalter ermöglicht es Nutzerinnen und Nutzern, eine große Auswahl an Bibliotheken zu durchsuchen und zu installieren. Bibliotheken erweitern die Grundfunktionen von Arduino und erleichtern Aufgaben wie:

- das Steuern von Servomotoren,
- das Auslesen von Daten bestimmter Sensoren oder
- die Arbeit mit Modulen wie WLAN.

Diese Bibliotheken enthalten vorgefertigten Code, um bestimmte Hardware-Komponenten anzusteuern und komplexe Aufgaben zu vereinfachen. Dadurch wird die Entwicklung von Arduino-Projekten schneller und effizienter – wie in 2.10 dargestellt.



Abbildung 2.10.: Library Manager

## 2.4. Integration und Verwendung einer benutzerdefinierten Bibliothek in der Arduino IDE

Bei der Entwicklung von eingebetteten Systemen auf der Arduino-Plattform sind Bibliotheken unerlässlich, um komplexe Funktionen zu abstrahieren und zu vereinfachen. Eine benutzerdefinierte Bibliothek ist besonders dann sinnvoll, wenn eine bestimmte Funktionalität oder eine spezifische Hardware-Schnittstelle häufig in verschiedenen Projekten verwendet wird.

Dieser Abschnitt beschreibt den Ablauf zum Erstellen, Installieren und Verwenden einer eigenen Bibliothek innerhalb eines Arduino-Projekts - in der Arduino Integrated Development Environment (IDE).

### 2.4.1. Erstellen der Bibliotheksdateien

Eine gut strukturierte, benutzerdefinierte Arduino-Bibliothek besteht typischerweise aus mindestens zwei zentralen Komponenten:

**Header-Datei (.h):** Diese Datei enthält die Deklarationen von Klassen, Funktionen und Variablen, die die Schnittstelle (Interface) der Bibliothek definieren. Sie stellt die notwendigen Definitionen bereit, damit externe Programme die Funktionen und Klassen verwenden können.

**Quellcode-Datei (.cpp):** Diese Datei enthält die tatsächliche Implementierung der in der Header-Datei deklarierten Funktionen und Methoden. Sie definiert das Verhalten der Funktionen und Klassen.

Die Struktur der Bibliothek ist so organisiert, dass eine klare Trennung zwischen Deklaration und Implementierung ihrer Funktionalitäten besteht.

### 2.4.2. Installation und Integration der Arduino IDE

Sobald die benutzerdefinierte Bibliothek erstellt wurde, muss sie korrekt installiert und in die Arduino IDE integriert werden, um in Projekten verwendet werden zu können. Um eine benutzerdefinierte Bibliothek in der Arduino IDE zu installieren, befolgen Sie diese Schritte:

1. Bibliotheksordner finden: Die Arduino IDE sucht nach Bibliotheken im Ordner `libraries`, der sich im Sketchbook-Verzeichnis befindet. Der typische Pfad zu diesem Verzeichnis lautet:

`C:/Users/<Benutzername>/Documents/Arduino/libraries`

- Benutzerdefinierten Bibliotheksordner kopieren: Kopieren Sie den Ordner ihrer Bibliothek, z.B. Nano33BLESenseLED, in das Verzeichnis **libraries**. Der vollständige Pfad sollte dann wie folgt aussehen:

**C:/Users/<Benutzername>/Documents/Arduino/libraries/Nano33BLESenseLED**

- Arduino IDE neu starten: Nachdem die Bibliothek in das richtige Verzeichnis kopiert wurde, starten sie die Arduino IDE neu, damit sie die neu hinzugefügte Bibliothek erkennt.

### 2.4.3. Verwendung symbolischer Verknüpfungen zur Bibliotheksintegration

In manchen Fällen kann es bequemer oder notwendig sein, die Bibliotheksdateien außerhalb des Standardbibliotheksverzeichnis zu speichern. Zum Beispiel, wenn die Bibliotheken in einem GitHub-Repository gespeichert sind oder zur besseren Versionskontrolle, kann der Befehl `mklink` verwendet werden, um eine symbolische Verknüpfung zu erstellen.

Dies ermöglicht es der Arduino IDE, auf die Bibliotheksdateien zuzugreifen, als ob sie sich im Standardverzeichnis befinden, ohne dass die Dateien dupliziert werden müssen.

### 2.4.4. Windows-Tool **mklink** zum Erstellen symbolischer Verknüpfungen

Symbolische Verknüpfungen ermöglichen es, Bibliotheken an einem anderen Speicherort abzulegen, während sie von der Arduino IDE trotzdem so behandelt werden, als befänden sie sich im Standardverzeichnis **libraries**.

Dies kann besonders nützlich sein in versionierten Umgebungen, z.B. bei der Verwendung von Git, oder um Bibliotheken besser zu organisieren, ohne Dateien zu duplizieren.

Schritte zum Erstellen einer symbolischen Verknüpfung:

- Öffnen der Eingabeaufforderung mit Administratorrechten:** Drücken Sie **Win + S** und suchen Sie nach `cmd`, klicken Sie dann auf **Eingabeaufforderung** und wählen Sie **Als Administrator ausführen** wie in 2.11 gezeigt.

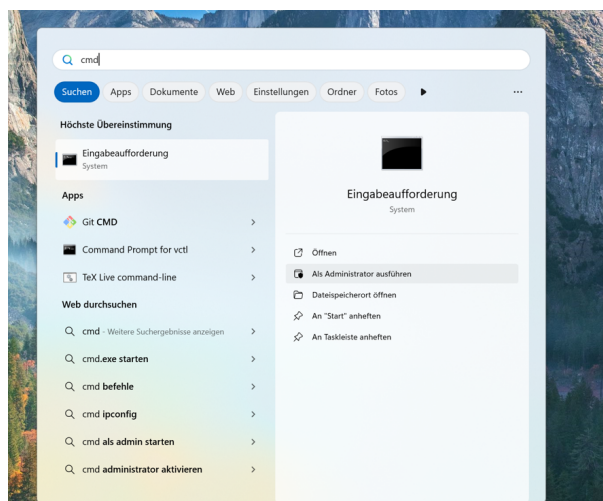


Abbildung 2.11.: Eingabeaufforderung mit Administrator Rechten

2. **Führen Sie den Befehl `mklink` aus:** Der Befehl zum Erstellen einer symbolischen Verknüpfung lautet: `mklink /D TTargetPathSourcePath`

- **TargetPath:** Der Ort, an dem die Arduino IDE die Bibliothek erwartet – also im Ordner `libraries`.
- **SourcePath:** Der tatsächliche Speicherort der Bibliothek.

In diesem Beispiel befindet sich die Bibliothek unter:

`C:/.../.../.../GitHub/241031ArduinoNano33BLESense/report/Code/Nano33BLESense/Nano33BLESenseLED`

Der Zielpfad im libraries-Ordner der Arduino IDE lautet:

`C:/.../.../.../Arduino/libraries/Nano33BLESenseLED`

Der vollständige Command ist in 2.12 zu sehen.

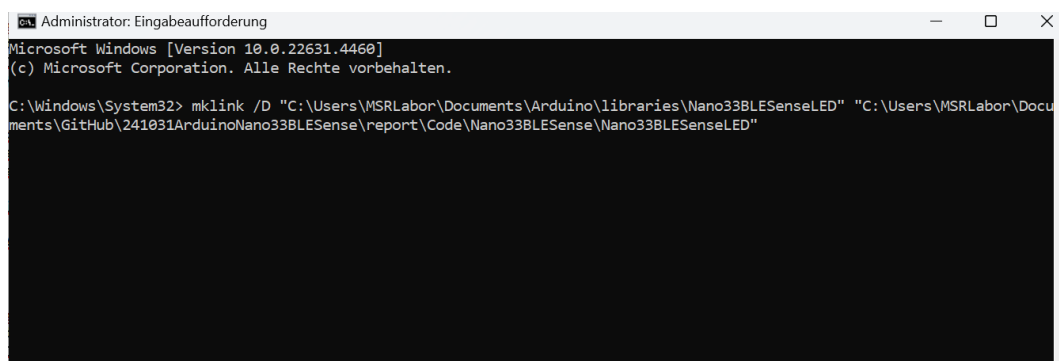


Abbildung 2.12.: Eingabeaufforderung

**Überprüfen der symbolischen Verknüpfung:** Nach dem Ausführen des Befehls navigieren Sie zum Ordner `libraries`. Dort sollte ein Ordner mit dem Namen `Nano33BLESenseLED` vorhanden sein, der als symbolische Verknüpfung fungiert und auf den tatsächlichen Speicherort der Bibliothek verweist.

#### 2.4.5. Überprüfen der Verfügbarkeit der Bibliothek in der Arduino IDE

Um sicherzustellen, dass die benutzerdefinierte Bibliothek erfolgreich integriert wurde und von der Arduino IDE erkannt wird, befolgen Sie diese Schritte:

1. Arduino IDE neu starten: Falls die Arduino IDE während der Installation der Bibliothek oder nach dem Erstellen der symbolischen Verknüpfung geöffnet war, schließen und öffnen Sie sie erneut. Dadurch wird die Liste der verfügbaren Bibliotheken neu geladen.
2. Bibliotheksmenü überprüfen: Navigieren Sie zu `Sketch` gehen Sie zu `Include Library` und wählen Sie die Bibliothek `Nano33BLESenseLED` aus, wie in 2.13 gezeigt.
3. Überprüfung in der Liste: Wenn die Bibliothek in der Liste erscheint, bedeutet das, dass sie von der IDE erfolgreich erkannt wurde. Falls nicht, überprüfen Sie nochmals die Verzeichnisstruktur und die symbolische Verknüpfung, um sicherzustellen, dass sie korrekt eingerichtet sind.



4. Testprogramm kompilieren: Schreiben Sie ein einfaches Testprogramm, das Funktionen oder Klassen aus der Bibliothek verwendet. Kompilieren Sie den Sketch, um sicherzustellen, dass keine Fehler auftreten – das bestätigt, dass die Bibliothek erfolgreich integriert wurde.

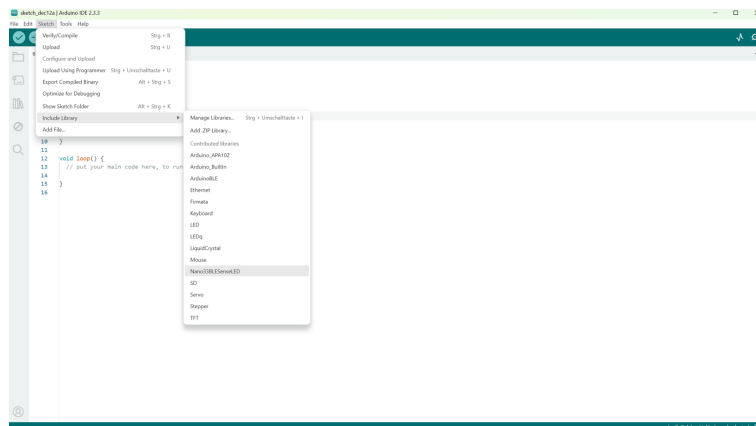


Abbildung 2.13.: Einbinden der Nano33BLESenseLED Bibliothek

Sobald das Programm ohne Fehler kompiliert, ist die Bibliothek einsatzbereit und kann bedenkenlos in Arduino-Projekten verwendet werden.

### 2.4.6. Serieller Monitor

Der Serieller Monitor kann über folgende Schritte aufgerufen werden: **Tool** **Serial Monitor**

Der Serieller Monitor ist ein Werkzeug, das es ermöglicht, Daten vom Board anzuzeigen, zum Beispiel mithilfe des Befehls `Serial.print()`.

Früher wurde dieses Tool in einem separaten Fenster geöffnet, ist aber jetzt in den Editor integriert, was es erleichtert, mehrere Instanzen gleichzeitig auf dem Computer auszuführen.

Der Serieller Monitor ist in 2.14 dargestellt.

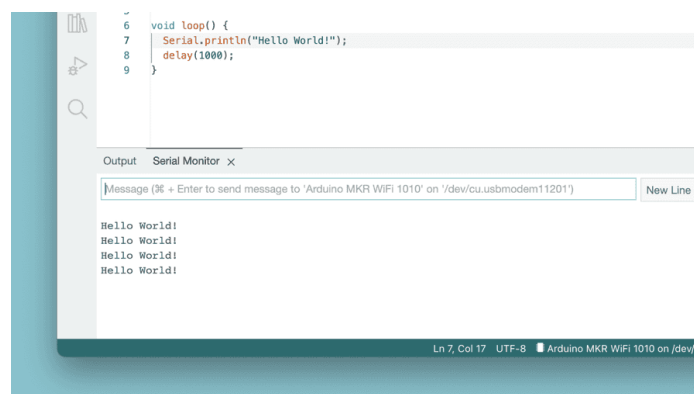


Abbildung 2.14.: Serieller Monitor

### 2.4.7. Serieller Plotter

Das Werkzeug **Serial Plotter** eignet sich hervorragend zur Visualisierung von Daten in Form von Diagrammen, z.B. zur Überwachung von Spannungsspitzen.

Es ermöglicht das gleichzeitige Beobachten mehrerer Variablen und bietet die Möglichkeit, bestimmte Datentypen gezielt ein- oder auszublenden.

## 2.5. Beispiele

Ein wesentlicher Bestandteil der Arduino Dokumentation ist die Sammlung von Beispielsketches, die mit verschiedenen Bibliotheken gebündelt sind.

Diese Beispiele zeigen die praktische Anwendung von Bibliotheksfunktionen und veranschaulichen deren typische Einsatzmöglichkeiten und Hauptfunktionen. Auch Bibliotheken, die mit bestimmten Board-Paketen geliefert werden, können eigene Beispielsketches enthalten.

Um auf diese Beispielsketches zuzugreifen – egal ob sie manuell installierten Bibliotheken oder mitgelieferten Board-Paketen zugeordnet sind – navigiere in der Arduino IDE zu: **File** » **Examples**, und wähle dann die gewünschte Bibliothek aus der Liste aus. Beispielsweise erscheint beim Anschluss eines UNO R4 WiFi-Boards eine Liste mit bezugsspezifischen Beispielen. Ein Beispielsketch mit einer vorgeladenen Tetris-Animation kann aufgerufen werden über: **File** » **Examples** » **LED Matrix** » **MatrixIntro** und auf das angeschlossene Board hochgeladen werden.

Dieses Beispiel zeigt, wie der mitgelieferte Code des Boards in der Praxis genutzt werden kann und hilft den Nutzern, verschiedene Hardwarefunktionen direkt zu steuern und kennenzulernen.

In Abbildung 2.15 wird die Beispielliste angezeigt, wenn ein UNO R4 WiFi-Board mit dem Computer verbunden ist.

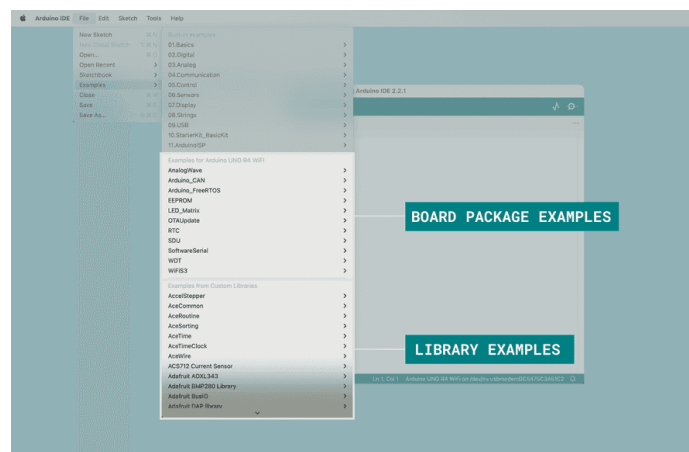


Abbildung 2.15.: Beispiele

## 2.6. Debuggen

Das Tool **Debugger** in der Arduino IDE 2.3.3 ist dafür konzipiert, Programme beim Testen und Debuggen zu unterstützen, indem es ermöglicht, die Codeausführung auf kontrollierte Weise Schritt für Schritt durchzugehen.

Dieses Tool erlaubt es den Nutzern, Haltepunkte zu setzen, den Code Zeile für Zeile durchzugehen und Variablen zu inspizieren, um Probleme wie Logikfehler oder falsche Variablenwerte zu identifizieren.

Der Debugger ermöglicht es, die Ausführung an bestimmten Punkten im Programm anzuhalten, um das Verhalten des Programms und den Speicherzustand im Detail zu untersuchen.

Diese Funktion verbessert den Entwicklungsprozess, indem sie es erleichtert, Fehler zur Laufzeit zu finden und zu beheben, anstatt sich ausschließlich auf print-Anweisungen oder Versuch-und-Irrtum zu verlassen 2.16.

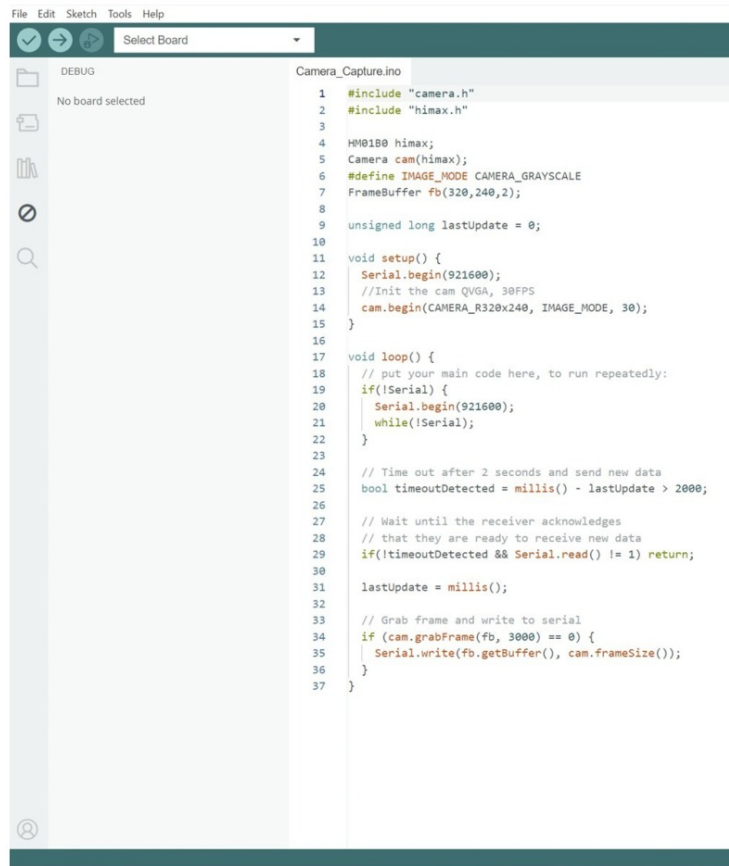


Abbildung 2.16.: Beispiel Debuggen

## 2.7. Autovervollständigung

Die Autovervollständigung ist eine zentrale Funktion der Arduino IDE Version 2, die das Programmieren erleichtert, indem sie Funktionen und Elemente aus der Arduino-API vorgeschlägt. Diese Funktion hilft dabei, Code schneller und genauer zu schreiben, was die Effizienz und Genauigkeit verbessert.

Damit die Autovervollständigung funktioniert, muss in der Arduino IDE das verwendete Board ausgewählt sein. Dadurch erkennt der Editor die richtigen Funktionen und Bibliotheken für das jeweilige Board und kann präzise Vorschläge machen 2.17.

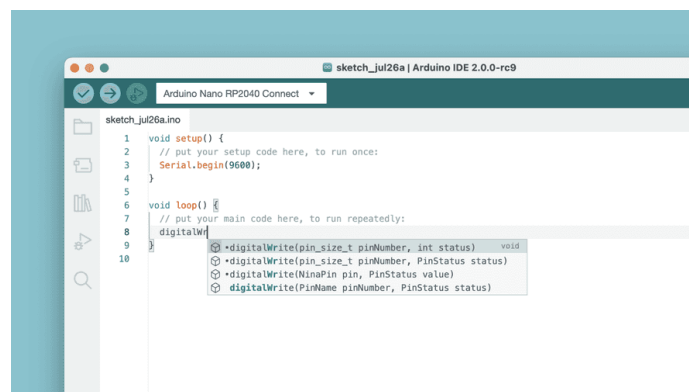


Abbildung 2.17.: Autovervollständigung

---

## 2.8. Bibliotheken und Modularität

Die Arduino IDE erlaubt es, Funktionen in Bibliotheken auszulagern und wiederzuverwenden. Viele Sensoren oder Module wie DHT11, LCD oder Motorshields bringen eigene Bibliotheken mit, die über die Bibliotheksverwaltung eingebunden werden können [Smi11]:

```
#include <ArduinoBLE.h>
```

Die Verwendung von Bibliotheksfunktionen ermöglicht es, größere Programme in modulare Teilfunktionen zu gliedern und dadurch Struktur sowie Wartbarkeit deutlich zu verbessern. So kann beispielsweise mit der Funktion `BLE.setLocalName()` ein Bluetooth-Name für die Applikation festgelegt werden, ohne dass die zugrunde liegenden Befehle und Abläufe im Detail bekannt sein müssen.

## 3. Arduino Web Editor

### 3.1. Using the Arduino Web Editor

The Arduino Web Editor allows you to write code and upload sketches to any official Arduino board directly from your web browser (Chrome, Firefox, Safari and Edge). [Ard24]

This IDE (Integrated Development Environment) is part of Arduino Create, an online platform that enables developers to write code, access tutorials, configure boards, and share projects. Designed to provide users with a continuous workflow, Arduino Create connects the dots between each part of a developer's journey from inspiration to implementation. Meaning, you now have the ability to manage every aspect of your project right from a single dashboard.

The Arduino Web Editor is hosted online, therefore it is always be up-to-date with the latest features and support for new boards. This IDE lets you write code and save it to the Cloud, always backing it up and making it accessible from any device. It automatically recognizes any Arduino board connected to your PC, and configures itself accordingly.

All you need to get started is an Arduino account. The following steps can guide you to start using the Arduino Web Editor: [Ard24]

#### 3.1.1. Using the online IDE

After logging in, you are ready to start using the Arduino Web Editor. The web app is divided into three main columns. 3.1

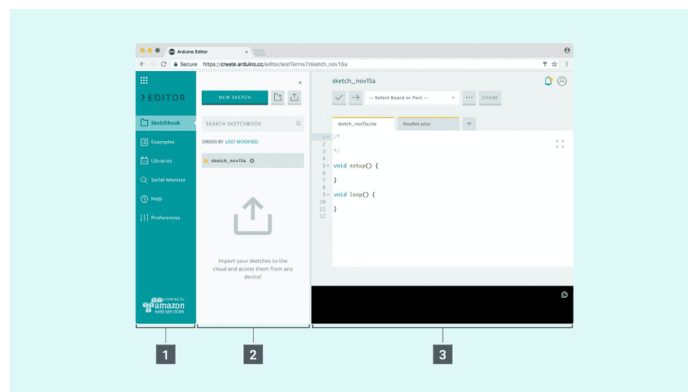


Abbildung 3.1.: Arduino Web Editor

The Arduino Web Editor's interface is as follows:

1. The first column lets you navigate between:
  - **Your Sketchbook:** a collection of all your sketches (a sketch is a program you upload on your board).
  - **Examples:** read-only sketches that demonstrate all the basic Arduino commands (built-in tab), and the behavior of your libraries (from the libraries tab).

- **Libraries:** packages that can be included to your sketch to provide extra functionalities.
  - **Serial monitor:** a feature that enables you to monitor, receive and send data to and from your board via the USB cable.
  - **Help:** helpful links and a glossary about Arduino terms.
  - **Preferences:** options to customize the look and behavior of your editor, such as text size and color theme.
2. The second column views the content of the chosen option.
  3. The third column, the code area, is the one you will use the most. Here, you can write code, verify it and upload it to your boards, save your sketches on the Cloud, and share them with anyone you want.

Now that we are all set up, let's try to make the board blink!

1. Connect your Arduino or Genuino board to your computer. Boards and serial ports are auto-discovered and selectable in a single dropdown. Pick the Arduino/Genuino board you want to upload to from the list at the top of the third column.[Ard24]
2. Let's try with an example: Choose Examples on the menu on the left (first column), then Basic and Blink. The Blink sketch is now displayed in the code area.
3. To upload it to your board, press the "Upload" button near the dropdown menu. While the code is verifying and uploading, a "BUSY" label replaces the upload button. If the upload is successful, the message "Success: done uploading" will appear in the bottom output area.
4. Once the upload is complete, you should see on your board the yellow LED with an L next to it start blinking. You can adjust the speed of blinking by changing the delay number in the parenthesis to 100, and upload the Blink sketch again. Now the LED should blink much faster ??.



Abbildung 3.2.: finding-an-example

## 3.2. Arduino Libraries

The process of setting up libraries on the online IDE (Arduino Cloud Editor) is quite similar to the offline one: [arduinoLibraries:2025; arduinoCloud:2025]

1. Login to the Arduino Cloud.

2. Create or open a sketch.
3. Open the LLibrariesttab from the left menu, and search for libraries. The list displays read-only libraries, authored and maintained by the Arduino team and its partners.
4. When you find the library, you can add it to your sketch by selecting the "Include" button. You can also see the related examples, and select a specific version, if available.
5. If you can't find a specific library on the list, you can search every existing library through the search bar. You also have the option to add them to your favorites list by clicking on the star next to the library you want. Once you star a library, you can view it under the ffavoritesttab and use its examples (if available).

### 3.3. Data Security

- **Encryption:** The web version of Arduino IDE should use strong encryption protocols to secure data transmitted between the user's browser and the server, preventing unauthorized access.
- **Secure Storage:** User data, including login credentials and project files, should be encrypted at rest to protect against unauthorized access or tampering.

#### 3.3.1. Authentication and Authorization

- **Strong Password Policies:** Enforce password complexity and offer multi-factor authentication to enhance account security.
- **Role-Based Access Control:** Implement role-based access to ensure users only access necessary resources, preventing unauthorized access to sensitive data.

#### 3.3.2. Secure Code Execution

- **Sandboxing:** Execute user-uploaded code in a sandboxed environment to restrict access to system resources and prevent malicious actions.
- **Code Validation:** Validate user code to detect and prevent vulnerabilities like code injection or buffer overflows before execution.

#### 3.3.3. Protection Against Malware

- **Code Scanning:** Use code scanning to detect and block malicious code by analyzing for malware signatures or suspicious patterns.
- **Antivirus Integration:** Integrate antivirus solutions to scan uploaded files for viruses or malicious content before execution.

#### 3.3.4. Privacy Policies

- **Transparency:** Maintain clear privacy policies outlining data collection, storage, usage, retention periods, and third-party sharing practices.
- **User Consent:** Inform users about privacy practices and obtain explicit consent for data processing, allowing them to review and modify privacy settings.

### 3.3.5. Regular Updates and Maintenance

- **Patch Management:** Regularly update software components, libraries, and dependencies to address security vulnerabilities promptly.
- **Security Audits:** Conduct regular security audits and penetration testing to identify and remediate potential weaknesses.

### 3.3.6. User Awareness

- **Security Education:** Provide resources on secure coding practices to help users avoid vulnerabilities like XSS, SQL injection, and insecure object references.
- **Security Alerts:** Notify users about security-related issues, such as suspicious login attempts or unauthorized access.



## **Teil II.**

# **Hardware**



## **4. Arduino Uno**

### **4.1. Was ist ein Arduino?**

### **4.2. Arduino Uno**

Beschreibung mit Tikz Bild

### **4.3. Sensoren**

- 
- 
- 

#### **4.3.1. Testtaster**

Testcode

Code

### **4.4. Module**

- 
- 
- 
- 

### **4.5. PINs**

- 
- 
- 
- 

### **4.6. Beispiel Code**

#### **4.6.1. Manual**

#### **4.6.2. Code**



## 5. CNCShieldV3

### 5.1. CNC Shields

General description  
cite books

### 5.2. CNCShieldV3

cite board

### 5.3. Specification

- Cite the data sheet
- Extract te information from the data sheet
- Circuit Diagram

### 5.4. Library

#### 5.4.1. Description

#### 5.4.2. Installation

- data types
- data structure
- data quantity
- data quality

#### 5.4.3. Functions

### 5.5. Example

In this section, a simple example is described in detail, which demonstrates how the library supports the sensor/actor.

#### 5.5.1. Manual

#### 5.5.2. Inside the Example

#### 5.5.3. Code

#### 5.5.4. Files

### 5.6. Calibration

cite method

## **5.7. Simple Code**

## **5.8. Simple Application**

## **5.9. Tests**

### **5.9.1. Simple Function Test**

### **5.9.2. Test all Functions**

## **5.10. Further Readings**

## 6. Sensor: Endschalter

### 6.1. Einleitung: Positionssensoren

In der industriellen Fertigungssteuerung nimmt die Erfassung von Zuständen eine zentrale Rolle ein. Gemäß der Fachliteratur wandelt ein Sensor (lat. sensus, der Sinn) „eine physikalische Größe (z. B. Kraft oder Temperatur) mit Hilfe eines physikalischen Effekts in ein elektrisches Signal um“ [Her+12, S. 433]. Speziell im Maschinenbau wird bei der Steuerung zwischen zwei grundlegenden Messverfahren unterschieden: Der „Positionserfassung durch Schalter“ und der „Positionserfassung durch Wegbeobachtung“. [Her+12, S. 435]

Bevor der Fokus auf kontaktbehaftete Schalter gelegt wird, sind folgende berührungslose Sensoren (Sensorschalter) als moderner Industriestandard zu nennen:

- Induktive Sensoren: Erfassen metallische Elemente „ohne Kontakt zwischen der Bewegung und dem Sensor“ [Her+12, S. 436].
- Kapazitive Sensoren: Nutzen die Beeinflussung eines elektrischen Feldes zur Kapazitätsänderung [Her+12, S. 438].
- Optische Sensoren: Arbeiten mit Infrarot-Lichtstrahlen zur Objekterkennung (z. B. Lichttaster oder Lichtschranken) [Her+12, S. 440].
- Ultraschall-Sensoren: Basieren auf der Messung der Laufzeit von Schallimpulsen [Her+12, S. 443].

### 6.2. Definition und Funktion: Elektromechanischer Endschalter

Ein elektromechanischer Endschalter ist ein kontaktbehafteter Positionssensor. Ein solcher Schalter „hat die Aufgabe, das Erreichen einer bestimmten Position (Endlage) zu melden oder einen Schaltvorgang auszulösen“ [Her+12, S. 435]. Dabei wird durch das Betätigen eines mechanischen Elements, wie beispielsweise eines Hebels oder Tasters, ein elektrischer Kontakt geöffnet oder geschlossen [Her+12, S. 120–122].

Technische Spezifikationen (Beispiel Creality): Bei dem vorliegenden Bauteil handelt es sich um einen „originalen Endschalter von Creality 3D, der bei fast allen FDM Druckern von Creality 3D verbaut ist“ [3dj].

### 6.3. Specification

- Kompatibilität: Geeignet für die X-, Y- oder Z-Achse von Modellen wie der Ender-3 oder CR-10 Serie [3dj][Bas].
- Hersteller: Creality
- Hersteller SKU: 4004180004
- Gewicht: 7.3g
- Abmaße: 28mm x 20mm x 9mm

- Auslöseweg: 4mm

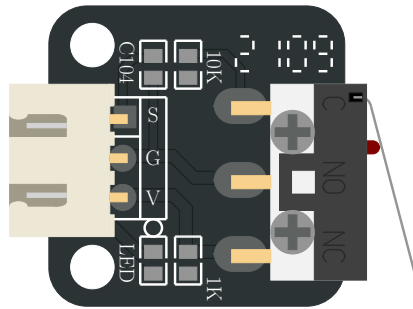


Abbildung 6.1.: Darstellung des im Projekt verwendeten Endschalters

## 6.4. Verwendung und Schaltfunktionen

Mechanische Endschalter werden zur Überwachung von Bewegungsabläufen eingesetzt. Dabei können verschiedene logische Funktionen realisiert werden:

- Schließer: Der Kontakt wird bei Betätigung geschlossen.
- Öffner: Der Kontakt wird bei Betätigung unterbrochen.
- Antivalenter Sensor: Dieser verfügt über zwei Ausgänge, wobei „stets ein Kontakt geschlossen und der andere offen“ ist [Her+12, S. 435]. Dies ermöglicht neben der reinen Schaltfunktion auch eine „Fehlererkennung und Diagnose durch die SPS“ [Her+12, S. 435–436].



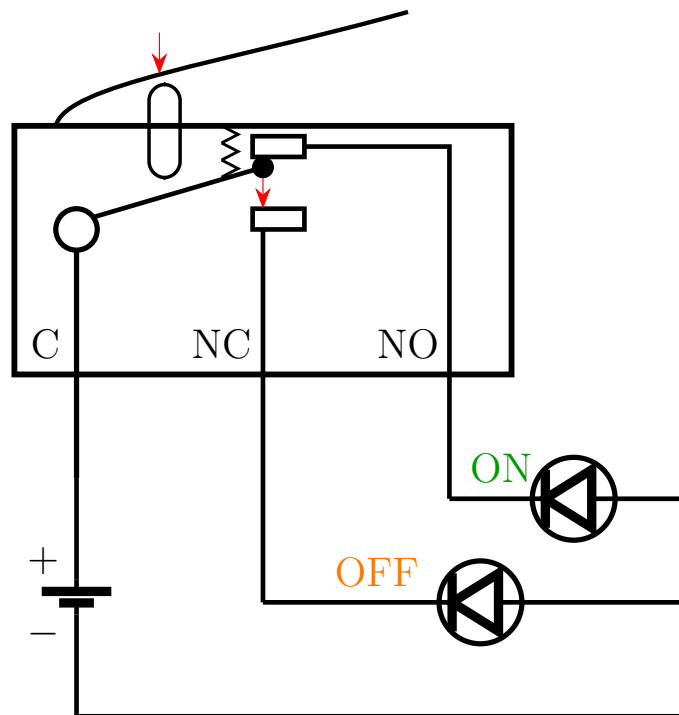


Abbildung 6.2.: Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO

Haupteinsatzgebiete:

- Endstopp-Erkennung: Schutz der Maschine vor dem Überfahren der mechanischen Grenzen.
- Referenzschalter (Homing): Der Schalter dient als „Referenzschalter“, um den Nullpunkt einer Achse (z. B. beim 3D-Drucker) zu kalibrieren [Bas].

## 6.5. Analyse: Vor- und Nachteile

**Vorteile:**

- Wirtschaftlichkeit: Sehr kostengünstig (ca. 4-5 € pro Stück) [3DJake, Bastelgarage].
- Einfachheit: Direkte Erzeugung eines binären Signals ohne aufwändige Auswerteelektronik.
- Robustheit gegen EMV: Unempfindlich gegenüber elektromagnetischen Störungen, da kein Oszillator wie bei induktiven Sensoren benötigt wird.

**Nachteile:**

- Mechanischer Verschleiß: Durch den physischen Kontakt unterliegen die Bauteile einer Abnutzung.
- Geschwindigkeit: Mechanische Kontakte können „prellen“ und sind langsamer als elektronische Lösungen [Her+12, S. 444].
- Technologische Verbreitung: Obwohl immer noch sehr viele mechanische Endschalter eingesetzt werden, ist der berührungslose Endschalter (Sensorschalter) in nahezu allen Bereichen als Standard-Bauelement zu finden [Her+12, S. 435].

## 6.6. Ausblick: Intelligente und vernetzte Sensorik

Die Zukunft der Positionserfassung liegt in der bidirektionalen Kommunikation. Ein moderner Standard ist hierbei IO-Link, welcher eine „feldbusunabhängige Kommunikation zwischen Sensoren und Aktoren und der Maschinensteuerung“ beschreibt [Her+12, S. 510].

Während klassische Endschalter nur „1 Bit“ an Informationen liefern (Ein/Aus), können intelligente Systeme über IO-Link zusätzlich Servicedaten übertragen, „die zur Einstellung oder Diagnose eines Devices führen“ [Her+12, S. 512].

Dies ermöglicht eine „vorbeugende Instandhaltung“, indem beispielsweise Verschleiß gemeldet wird, bevor es zum Maschinenausfall kommt [Her+12, S. 471, 512].

## 6.7. Library

Für die Verwendung des Endschalters werden keine zusätzlichen Bibliotheken benötigt. Es werden Funktionen aus der Arduino-Standardbibliothek verwendet:

- `pinMode()`: Konfiguriert einen Pin als Ein- oder Ausgang.
- `digitalRead()`: Liest den aktuellen Zustand eines digitalen Eingangs.
- `digitalWrite()`: Setzt einen digitalen Ausgang auf HIGH oder LOW.
- `Serial.begin()`: Initialisiert die serielle Kommunikation.
- `Serial.print()` / `Serial.println()`: Gibt Daten im Serial Monitor aus.
- `delay()`: Erzeugt eine zeitliche Verzögerung im Programmablauf.

## 6.8. Beispiel

Dieses Beispiel zeigt, wie der Endschalter über einen digitalen Eingang ausgelesen werden kann. Dabei wird der interne Pull-Up-Widerstand verwendet. Der Zustand des Schalters wird im Serial Monitor angezeigt und ändert sich beim Betätigen von HIGH auf LOW.

### 6.8.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND.

### 6.8.2. Erklärung des Beispiel-Codes

Nach der Initialisierung durch die Funktion `setup()` wird das Programm durch die Funktion `loop()` kontinuierlich ausgeführt. Diese Funktion stellt eine Endlosschleife dar, in der der Zustand des Endschalters bei jedem Durchlauf überprüft wird.[Ardj; Ardg]

Der verwendete Pin wird mit der Funktion `pinMode()` als Eingang oder Ausgang konfiguriert. [Ardi] Dabei wird der Modus `INPUT_PULLUP` verwendet, um den internen Pull-Up-Widerstand zu aktivieren.[Arda] Der interne Pull-Up-Widerstand sorgt dafür, dass am Eingang im Ruhezustand ein definiertes HIGH-Signal anliegt. Innerhalb der `loop()`-Funktion wird der Zustand des Endschalters mit der Funktion `digitalRead()` ausgelesen. Der gelesene Wert wird in einer Variablen gespeichert und über die serielle Schnittstelle ausgegeben.[Arde] Die Funktionen `Serial.print()` und `Serial.println()` werden verwendet, um den aktuellen Zustand des Eingangs im Serial

Monitor darzustellen.[Ardc] Durch die Verwendung des internen Pull-Up-Widerstands ergibt sich eine invertierte Logik. Im nicht betätigten Zustand des Endschalters liegt ein HIGH-Signal am Eingang an. Wird der Endschalter betätigt liegt ein LOW-Signal an. Zur besseren Lesbarkeit der Ausgabe wird eine Verzögerung mit der Funktion `delay()` erzeugt.[Ardd]

Listing 6.1.: Einfaches Beispiel zum Lesen eines mechanischen Endschalters

```

1000 /**
1001  * @file MicroSwitch_BasicRead.ino
1002  * @brief Basic example for reading a micro switch.
1003  */
1004 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
1005     switch. */
1006 /**
1007  * @brief Initializes the serial interface and the input pin.
1008  */
1009 void setup()
1010 {
1011     Serial.begin(115200);
1012     pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
1013 }
1014 /**
1015  * @brief Reads the switch state and prints it to the serial monitor.
1016  */
1017 void loop()
1018 {
1019     int switchState = digitalRead(MICRO_SWITCH_PIN);
1020
1021     Serial.print("Micro switch state: ");
1022     Serial.println(switchState);
1023
1024     delay(200);
1025 }

```

../Code/MicroSwitchBasicRead/MicroSwitchBasicRead.ino

## 6.9. Justage

Die Justage (auch "Justierung") ist eine "Tätigkeit, welche ein Messgerät in einen gebrauchstauglichen Betriebszustand versetzt. Sie kann manuell, halb automatisch oder automatisch erfolgen." [Hel, S. 17] Sie kann auch als das "abgleichen des Nullpunktes und der Kennliniensteigung" [Ber24, S. 17] verstanden werden. Da der verwendete Endschalter nur die analogen Werte OPEN = 0 und CLOSED = 1 detektieren kann, sodass keine Kennlinie im herkömmlichen Sinne vorliegt, liegt die anwendungsspezifische Justage-Aufgabe darin, den gesamten Endschalter physisch in seiner Position am Rahmengestell so auszurichten, dass er die entsprechende bewegliche Achse im Rahmen einer Referenzfahrt in ihrer Bewegung eingrenzt, aber gleichzeitig im Normalbetrieb den erforderlichen Bauraum nicht künstlich einschränkt.

## 6.10. Tests

### 6.10.1. Einfacher Funktions-Test

Im einfachsten Anwendungstest wird der Endschalter verwendet, um eine LED zu steuern. Hierbei wird der Zustand des Schalters ausgewertet und auf einen Ausgang übertragen. Wird der Endschalter betätigt, schaltet sich die LED ein. Im nicht betätigten Zustand bleibt die LED ausgeschaltet. Durch diesen Test kann die Funktion

des Sensors sowie die Verarbeitung des Signals im System überprüft werden.

### 6.10.2. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND. Die LED wird an einen digitalen Ausgang (z. B. D13) angeschlossen.

Beim Betätigen des Endschalters wird die LED eingeschaltet, im nicht betätigten Zustand bleibt sie ausgeschaltet.

### 6.10.3. Einfacher Test-Code

Die Funktion `loop()` läuft dauerhaft und liest den Zustand des Mikroschalters ein. Da `INPUT_PULLUP` verwendet wird, bedeutet `LOW` = gedrückt. Ist der Schalter gedrückt, wird die LED eingeschaltet, sonst ausgeschaltet.

Listing 6.2.: An- und Ausschalten einer LED mittels Endschalter

```

1000 /**
1001  * @file MicroSwitch_LEDEExample.ino
1002  * @brief Simple application example using a micro switch and an LED.
1003  */
1004
1005 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
1006     switch. */
1007 const int LED_PIN = 13;          /**< Digital output pin for the built-in
1008     LED. */
1009
1010 /**
1011  * @brief Initializes the input and output pins.
1012  */
1013 void setup()
1014 {
1015     pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
1016     pinMode(LED_PIN, OUTPUT);
1017 }
1018
1019 /**
1020  * @brief Switches the LED depending on the micro switch state.
1021  */
1022 void loop()
1023 {
1024     int switchState = digitalRead(MICRO_SWITCH_PIN);
1025
1026     if (switchState == LOW)
1027     {
1028         digitalWrite(LED_PIN, HIGH);
1029     }
1030     else
1031     {
1032         digitalWrite(LED_PIN, LOW);
1033     }
1034 }

```

../Code/MicroSwitchLEDEExample/MicroSwitchLEDEExample.ino

## 6.11. Einfache Anwendung

Das Programm führt eine Referenzfahrt mit einem Schrittmotor aus.

Im `setup()` werden die Pins für den Treiber und den Endschalter eingerichtet. Danach wird der Motortreiber aktiviert und die Referenzfahrt gestartet.

Die Funktion `oneStep()` erzeugt einen einzelnen Schritt des Motors.

Mit `moveSteps()` kann der Motor eine festgelegte Anzahl an Schritten in eine bestimmte Richtung fahren.

In `homeAxis()` fährt der Motor so lange, bis der Endschalter gedrückt wird. Danach wartet das Programm kurz und fährt den Motor einige Schritte zurück.

### 6.11.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters wird an D2 angeschlossen, der zweite Anschluss liegt auf GND. Der Schrittmotortreiber wird über die Steuerpins STEP und DIR an digitale Ausgänge des Mikrocontrollers angeschlossen (z. B. STEP an D3 und DIR an D4).

Listing 6.3.: Referenzfahrt einer Achse mit Endschalter

```

1000 /**
1001  * @file referenzfahrt.ino
1002  * @brief Homing routine using a stepper motor, A4988 driver, and
1003  *        endstop.
1004  */
1005
1006 /** @brief STEP pin of the driver */
1007 const int stepPin = 2;
1008
1009 /** @brief Direction pin of the driver */
1010 const int dirPin = 5;
1011
1012 /** @brief Enable pin of the driver */
1013 const int enPin = 8;
1014
1015 /** @brief Endstop input pin */
1016 const int endstopPin = 9;
1017
1018 /** @brief Step delay in microseconds */
1019 const int stepDelayUs = 1000;
1020
1021 /** @brief Number of steps for the backoff movement */
1022 const int backoffSteps = 50;
1023
1024 /** @brief Initializes the hardware and starts the homing routine */
1025 void setup() {
1026     pinMode(stepPin, OUTPUT);
1027     pinMode(dirPin, OUTPUT);
1028     pinMode(enPin, OUTPUT);
1029     pinMode(endstopPin, INPUT_PULLUP);
1030
1031     Serial.begin(115200);
1032
1033     digitalWrite(enPin, LOW);
1034     digitalWrite(dirPin, LOW);
1035
1036     homeAxis();
1037 }
1038
1039 /** @brief Main loop (unused in this example) */
1040 void loop() {
1041
1042 }
1043
1044 /** @brief Executes a single motor step */
1045 void oneStep() {
1046     digitalWrite(stepPin, HIGH);
1047     delayMicroseconds(stepDelayUs);
1048     digitalWrite(stepPin, LOW);
1049     delayMicroseconds(stepDelayUs);
1050 }

```

```
1050 /**  
1051  * @brief Moves the motor by a defined number of steps  
1052  * @param steps Number of steps to execute  
1053  * @param dir Direction of movement (LOW/HIGH)  
1054  */  
1055 void moveSteps(int steps, bool dir) {  
1056     digitalWrite(dirPin, dir);  
1057     for (int i = 0; i < steps; i++) {  
1058         oneStep();  
1059     }  
1060 }  
  
1062 /** @brief Performs the homing sequence and backoff movement */  
1063 void homeAxis() {  
1064     while (digitalRead(endstopPin) == HIGH) {  
1065         oneStep();  
1066     }  
  
1068     delay(200);  
1069     moveSteps(backoffSteps, HIGH);  
1070 }
```

../Code/referenzfahrt/referenzfahrt.ino

## 6.12. Weiterführende Literatur

- Zielke, Dirk: *Mikrosysteme*. Springer, 2025. [Zie25] Das Buch behandelt die Grundlagen und Anwendungen von Mikrosystemen und bietet einen Überblick über Aufbau, Funktion und Einsatz moderner Sensorsysteme.
- Tränkler, Hans-Rolf und Reindl, Leo: *Sensortechnik*. Springer, 2014. [TR14] Das Buch behandelt grundlegende Prinzipien der Sensortechnik und beschreibt verschiedene Sensorarten sowie deren Einsatz in technischen Systemen.

## 7. Druckknopf

### 7.1. General

Ein elektrischer Druckknopf, auch Drucktaster genannt, ist ein elektromechanisches Bedienelement, mit dem ein Stromkreis durch Drücken geöffnet oder geschlossen wird. Je nach Ausführung kann er als Schließer oder Öffner arbeiten. Nach dem Loslassen kehrt ein Taster wieder in seine Ausgangsstellung zurück. [Rs2]

### 7.2. Joy IT

Beim Joy-IT Button-Micro handelt es sich um einen kompakten Micro-Arcadebutton mit Mikroschalter. Er besteht aus Nylon, ist in mehreren Farben erhältlich und für eine Belastungsgrenze von 12 V ausgelegt.[Joy]

### 7.3. Spezifikationen

- Abmessungen: Ø 32.8 x 29.9 mm
- Belastungsgrenze: 10 mA / 12 V DC [Joy]
- Schaltkreis

#### 7.3.1. Functions

Für die Verwendung eines einfachen Druckknopfes werden keine zusätzlichen Bibliotheken benötigt. Die Abfrage kann direkt mit den Standardfunktionen der Arduino IDE erfolgen.

### 7.4. Beispiel Code

#### 7.4.1. Manual

Der Taster wird mit einem Anschluss an GND und mit dem anderen an Pin 2 angeschlossen. Die LED bekommt einen 220 Ohm oder 330 Ohm Vorwiderstand. Der lange LED-Anschluss ist die Anode (+) und kommt über den Widerstand an Pin 13. Der kurze LED-Anschluss ist die Kathode (-) und kommt an GND. Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()` legt fest, ob ein Pin als Eingang oder Ausgang verwendet wird. [Ardi]
- `INPUT_PULLUP` aktiviert den interne Pull-up-Widerstand des Arduino. [Arda]
- `digitalRead()` liest den Zustand des Druckknopfes aus. [Arde]
- `digitalWrite()` wird die LED ein- oder ausgeschalten. [Ardf]

## 7.4.2. Code

Listing 7.1.: Led mit Druckknopf

```

1000 /**
1001  * @file ledmitdruckknopf.ino
1002  * @brief Schaltet eine LED ein, solange ein Druckknopf gedrueckt wird.
1003  *
1004  * Der Druckknopf ist zwischen Pin 2 und GND angeschlossen.
1005  * Die LED ist mit einem 220 Ohm oder 330 Ohm Vorwiderstand an Pin 13
1006  * angeschlossen.
1007  * Der lange LED-Anschluss ist die Anode (+), der kurze Anschluss ist
1008  * die Kathode (-).
1009  */
1010
1011 const int druckknopfPin = 2;    ///< Pin, an dem der Druckknopf
1012      angeschossen ist
1013 const int ledPin = 13;          ///< Pin, an dem die LED angeschlossen ist
1014
1015 /**
1016  * @brief Wird einmal beim Start des Arduino ausgefuehrt.
1017  *
1018  * Hier werden die Pins als Ein- oder Ausgang festgelegt.
1019  */
1020 void setup() {
1021     pinMode(druckknopfPin, INPUT_PULLUP);    ///< Druckknopfeingang mit
1022         internem Pull-up
1023     pinMode(ledPin, OUTPUT);                 ///< LED-Pin als Ausgang
1024 }
1025
1026 /**
1027  * @brief Hauptprogramm, das dauerhaft wiederholt wird.
1028  *
1029  * Wenn der Druckknopf gedrueckt wird, wird die LED eingeschaltet.
1030  * Wird der Druckknopf losgelassen, wird die LED ausgeschaltet.
1031  */
1032 void loop() {
1033     int druckknopfStatus = digitalRead(druckknopfPin);    ///< Aktueller
1034         Zustand des Druckknopfes
1035
1036     if (druckknopfStatus == LOW) {
1037         digitalWrite(ledPin, HIGH);    ///< LED einschalten
1038     } else {
1039         digitalWrite(ledPin, LOW);    ///< LED ausschalten
1040     }
1041 }

```

../Code/ledmitdruckknopf/ledmitdruckknopf.ino

## 7.5. Einfacher Testcode

### 7.5.1. Manual

Der Taster wird mit einem Anschluss an GND und mit dem anderen an Pin 2 angeschlossen. Im seriellen Monitor der Arduino IDE wird dann angezeigt, ob der Druckknopf gedrückt ist oder nicht.

Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()`
- `INPUT_PULLUP`
- `digitalRead()`
- `Serial.begin()` Startet die serielle Verbindung zum Computer.[Ardb]



- `Serial.println()` Gibt den Wert 1 oder 0 im seriellen Monitor aus. [Ardc]
- `delay()` Wartet kurz, damit die Ausgabe lesbar bleibt. [Ardd]

### 7.5.2. Code

Listing 7.2.: Druckknopf Test

```

1000 /**
1001  * @file druckknopfrtest.ino
1002  * @brief Testet einen Druckknopf und gibt 0 oder 1 im seriellen Monitor
1003  *       aus.
1004  *
1005  * Der Druckknopf ist zwischen Pin 2 und GND angeschlossen.
1006  * Es wird der interne Pull-up-Widerstand verwendet.
1007 */
1008 const int druckknopfPin = 2; ///< Pin, an dem der Druckknopf
1009        angeschlossen ist
1010
1011 /**
1012  * @brief Wird einmal beim Start des Arduino ausgefuehrt.
1013  *
1014  * Der Druckknopf-Pin wird als Eingang mit internem Pull-up-Widerstand
1015  * festgelegt.
1016  * Die serielle Ausgabe wird gestartet.
1017 */
1018 void setup() {
1019     pinMode(druckknopfPin, INPUT_PULLUP); ///< Eingang mit internem Pull-
1020     up
1021     Serial.begin(115200);                ///< Startet die serielle
1022     Verbindung
1023 }
1024
1025 /**
1026  * @brief Hauptprogramm, das dauerhaft wiederholt wird.
1027  *
1028  * Gibt 1 aus, wenn der Druckknopf gedrueckt ist.
1029  * Gibt 0 aus, wenn der Druckknopf nicht gedrueckt ist.
1030 */
1031 void loop() {
1032     int druckknopfStatus = digitalRead(druckknopfPin); ///< Liest den
1033     Zustand des Druckknopf
1034
1035     if (druckknopfStatus == LOW) {
1036         Serial.println(1); ///< Druckknopf gedrueckt
1037     } else {
1038         Serial.println(0); ///< Druckknopf nicht gedrueckt
1039     }
1040
1041     delay(200); ///< Kurze Pause fuer eine lesbare Ausgabe
1042 }

```

../Code/druckknopftest/druckknopftest.ino

## 7.6. Einfache Anwendung

Interrupt? Für Pause oder Start und mehrere LEDS gehen nach einander an.

### 7.6.1. Manual

Erklärung: In diesem Programm werden drei Druckknöpfe verwendet: Start, Pause und Stop. Der Start-Druckknopf startet eine LED-Abfolge, bei der fünf LEDs nacheinander

aufleuchten. Der Pause-Druckknopf hält die Abfolge an und setzt sie beim erneuten Drücken fort. Der Stop-Druckknopf beendet die Abfolge und schaltet alle LEDs aus. Anschluss: Die drei Druckknöpfe werden jeweils mit einem Anschluss an GND angeschlossen. Die anderen Anschlüsse kommen an Pin 2, 3 und 4. Die LEDs werden jeweils mit 220 Ohm oder 330 Ohm Vorwiderstand an Pin 8 bis 12 angeschlossen. Der lange Anschluss der LED kommt zum Arduino-Pin, der kurze Anschluss zu GND. Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()`
- `digitalRead()`
- `digitalWrite()`
- `digitalWrite()`
- `millis()` Liefert die Zeit seit dem Start des Arduino in Millisekunden. [Ardh]
- `delay()`

## 7.6.2. Code

Listing 7.3.: Druckknöpfe Led Abfolge

```

1000 /**
1001  * @file druckknoepfedabfolge.ino
1002  * @brief Steuert eine LED-Abfolge mit drei Druckknoepfen.
1003  *
1004  * Mit dem Start-Druckknopf wird die LED-Abfolge gestartet.
1005  * Mit dem Pause-Druckknopf wird die Abfolge pausiert oder fortgesetzt.
1006  * Mit dem Stop-Druckknopf wird die Abfolge beendet und alle LEDs werden
1007  *   ausgeschaltet.
1008  */
1009
1010 const int startdruckknopf = 2; ///< Eingangspin fuer den Start-
1011     Druckknopf
1012 const int pausedruckknopf = 3; ///< Eingangspin fuer den Pause-
1013     Druckknopf
1014 const int stopdruckknopf = 4; ///< Eingangspin fuer den Stop-
1015     Druckknopf
1016
1017 const int ledPins[] = {8, 9, 10, 11, 12}; ///< Ausgangspins fuer die
1018     LEDs
1019 const int anzahlLeds = 5;                ///< Anzahl der
1020     angeschlossenen LEDs
1021
1022 bool programmLaeuft = false; ///< Gibt an, ob die LED-Abfolge aktiv ist
1023 bool programmPause = false; ///< Gibt an, ob die LED-Abfolge pausiert
1024     ist
1025
1026 int aktuelleLed = 0;                    ///< Nummer der aktuell
1027     eingeschalteten LED
1028 unsigned long letzteZeit = 0;           ///< Zeitpunkt der letzten LED-
1029     Umschaltung
1030 const unsigned long intervall = 500;    ///< Zeitabstand zwischen den LEDs
1031     in Millisekunden
1032
1033 bool letzterPauseStatus = HIGH; ///< Vorheriger Zustand des Pause-
1034     Druckknopfs
1035
1036 /**
1037  * @brief Wird einmal beim Start des Arduino ausgefuehrt.
1038  *
1039  * Die Druckknoepfe werden als Eingange mit internem Pull-up-Widerstand
1040  * festgelegt.

```

```

1030  * Die LED-Pins werden als Ausgaenge festgelegt.
1031  */
1032  void setup() {
1033      pinMode(startdruckknopf, INPUT_PULLUP); ///< Start-Druckknopf als
1034      Eingang
1035      pinMode(pausedruckknopf, INPUT_PULLUP); ///< Pause-Druckknopf als
1036      Eingang
1037      pinMode(stopdruckknopf, INPUT_PULLUP); ///< Stop-Druckknopf als
1038      Eingang
1039
1040      for (int i = 0; i < anzahlLeds; i++) {
1041          pinMode(ledPins[i], OUTPUT); ///< LED-Pin als Ausgang festlegen
1042      }
1043
1044      alleLedsAus();
1045  }
1046
1047  /**
1048  * @brief Hauptprogramm, das dauerhaft wiederholt wird.
1049  *
1050  * Es prueft die drei Druckknoepfe und steuert je nach Eingabe
1051  * den Ablauf der LEDs.
1052  */
1053  void loop() {
1054      if (digitalRead(startdruckknopf) == LOW) {
1055          programmLaeuft = true;
1056          programmPause = false;
1057      }
1058
1059      if (digitalRead(stopdruckknopf) == LOW) {
1060          programmLaeuft = false;
1061          programmPause = false;
1062          aktuelleLed = 0;
1063          alleLedsAus();
1064      }
1065
1066      bool aktuellerPauseStatus = digitalRead(pausedruckknopf); ///<
1067      Aktueller Zustand des Pause-Druckknopfs
1068
1069      if (letzterPauseStatus == HIGH && aktuellerPauseStatus == LOW) {
1070          programmPause = !programmPause;
1071          delay(200); ///< Einfache Entprellung des Druckknopfs
1072      }
1073
1074      letzterPauseStatus = aktuellerPauseStatus;
1075
1076      if (programmLaeuft && !programmPause) {
1077          ledAbfolge();
1078      }
1079  }
1080
1081  /**
1082  * @brief Schaltet alle LEDs aus.
1083  *
1084  * Alle LED-Pins werden auf LOW gesetzt.
1085  */
1086  void alleLedsAus() {
1087      for (int i = 0; i < anzahlLeds; i++) {
1088          digitalWrite(ledPins[i], LOW);
1089      }
1090  }
1091
1092  /**
1093  * @brief Laesst die LEDs nacheinander aufleuchten.
1094  *
1095  * Nach Ablauf des festgelegten Intervalls wird die aktuelle LED
1096  * ausgeschaltet
1097  * und die naechste LED eingeschaltet.
1098  */

```

```
void ledAbfolge() {  
1094   unsigned long aktuelleZeit = millis(); ///  
        Aktuelle Laufzeit des  
        Arduino in Millisekunden  
  
1096   if (aktuelleZeit - letzteZeit >= intervall) {  
1098       letzteZeit = aktuelleZeit;  
  
        alleLedsAus();  
1100       digitalWrite(ledPins[aktuelleLed], HIGH);  
  
1102       aktuelleLed++;  
  
1104       if (aktuelleLed >= anzahlLeds) {  
1106           aktuelleLed = 0;  
        }  
1108   }
```

../Code/druckknoepfedabfolge/druckknoepfedabfolge.ino

## 7.7. Further Readings

**Teil III.**

## **Domain Knowledge - Tools**



**Teil IV.**

**Development**





**Teil V.**

**Monitoring**



**Teil VI.**

**Verification - Evaluation -  
Conclusion**



**Teil VII.**

**Anhang**



## 8. Bill of Material

### 8.1. Mechanische Komponenten

Tabelle 8.1.: Mechanische Komponenten

| Bild | Menge          | Beschreibung                                               | Lieferant   | Preis (€) |
|------|----------------|------------------------------------------------------------|-------------|-----------|
| –    | 6 (4 benötigt) | Basisplatte für Sensor                                     | Creality 3D |           |
| –    | 6 (4 benötigt) | Sensor Gehäuse                                             | Creality 3D |           |
| –    | 2              | Zahnriemen 5,5 x 1200                                      |             |           |
| –    | 10             | Rollen (gelagert), außen 24 mm, innen 4,5 mm, Breite 11 mm | Creality 3D |           |
| –    | 4              | Führungsstange d=10 mm, l=525 mm                           | Creality 3D |           |
| –    | 4 (2 benötigt) | Führungsstange d=10 mm, l=645 mm                           |             |           |
| –    | 4              | Alu-Extrusionsprofil 20x20, l=554 mm                       |             |           |
| –    | 4              | Alu-Extrusionsprofil 20x20, l=498 mm                       |             |           |
| –    | 4              | Alu-Extrusionsprofil 20x20, l=510 mm                       |             |           |
| –    | 2              | Alu-Extrusionsprofil 20x20, l=514 mm                       |             |           |
| –    | 2              | Welle für Riemenantrieb d=8 mm                             |             |           |
| –    | 2              | Spindel d=8 mm, l=470 mm                                   |             |           |
| –    | 2              | Spindelmuffe                                               |             |           |
| –    | 2              | Flachspiralfeder                                           |             |           |
| –    |                | Befestigung für Führung                                    |             |           |

## 8.2. Elektronische Komponenten

Tabelle 8.2.: Elektronische Komponenten

| Bild | Menge          | Beschreibung              | Lieferant     | Preis (€) |
|------|----------------|---------------------------|---------------|-----------|
| –    | 1              | Arduino Uno R3            |               |           |
| –    | 1              | CNC Shield V3             |               |           |
| –    | 4              | Stepper Motor Driver      |               |           |
| –    |                | Jumper Kabel              |               |           |
| –    | 4              | Stepper Motor             | Creality 3D   |           |
| –    | 8 (6 benötigt) | Endschalter               | www.3djake.de | 4,99      |
| –    | 4 (4 benötigt) | Druckknopf                | reichelt.de   |           |
| –    | 1 (1 benötigt) | OLED Display 0,96"        | reichelt.de   | 15,20     |
| –    | 1 (1 benötigt) | MOSFET (IRLZ44N)          | reichelt.de   | 0,70      |
| –    | 1              | Netzteil                  |               |           |
| –    | 1              | Netzkabel (Computerkabel) |               |           |
| –    | 1              | Heizdraht (Nichrom)       |               |           |

## 8.3. Schrauben und Befestigungen

Tabelle 8.3.: Schrauben und Befestigungen

| Bild | Menge | Beschreibung                                                  | Lieferant   | Preis (€) |
|------|-------|---------------------------------------------------------------|-------------|-----------|
| –    |       | Zylinderschraube M5x20                                        | Creality 3D |           |
| –    |       | Linsenkopfschraube M5x18                                      | Creality 3D |           |
| –    |       | Zylinderschraube M4x8                                         | Creality 3D |           |
| –    |       | Spanplattenschraube / Holzschraube 4x13 Senkkopf Kreuzschlitz | Creality 3D |           |
| –    |       | Zylinderschraube M3x14                                        | Creality 3D |           |
| –    |       | Linsenkopfschraube M3x8                                       | Creality 3D |           |
| –    |       | Federring M5                                                  | Creality 3D |           |
| –    |       | Federring M4                                                  | Creality 3D |           |
| –    |       | Federring M3                                                  | Creality 3D |           |



# Literaturverzeichnis

- [3dj] *Creality Endschalter*. 2026. URL: <https://www.3djake.de/creality-3d-drucker-ersatzteile/endschalter> (besucht am 26.04.2026).
- [Arda] *Digital Pins - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/variablen/constants/inputOutputPullup/> (besucht am 25.04.2026).
- [Ardb] *Serial.begin() - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/funktionen/communication/serial/begin/> (besucht am 10.05.2026).
- [Ardc] *Serial.print() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/communication/serial/print/> (besucht am 25.04.2026).
- [Ardd] *delay() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/time/delay/> (besucht am 25.04.2026).
- [Arde] *digitalRead() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/digitalread/> (besucht am 25.04.2026).
- [Ardf] *digitalWrite() - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/funktionen/digital-io/digitalwrite/> (besucht am 10.05.2026).
- [Ardg] *loop() - Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/loop/> (besucht am 25.04.2026).
- [Ardh] *millis() - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/funktionen/time/millis/> (besucht am 10.05.2026).
- [Ardi] *pinMode() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/pinMode/> (besucht am 25.04.2026).
- [Ardj] *setup() - Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/setup/> (besucht am 25.04.2026).
- [Ard24] Arduino Documentation. *The Arduino Web Editor*. 2024. URL: <https://docs.arduino.cc/learn/starting-guide/the-arduino-web-editor/>.
- [Bas] *Creality Endschalter*. 2026. URL: <https://www.bastelgarage.ch/creality-endschalter-limit-switch> (besucht am 26.04.2026).
- [Ber24] H. Bernstein. *Messelektronik und Sensoren. Grundlagen der Messtechnik, Sensoren, analoge und digitale Signalverarbeitung*. 2. Aufl. Springer Vieweg Wiesbaden, 25. Jan. 2024. ISBN: 978-3-658-38929-1. DOI: <https://doi.org/10.1007/978-3-658-38929-1>.
- [FAD18] M. Fezari und A. Al Dahoud. "Integrated Development Environment "IDE" For Arduino". In: (Okt. 2018).

- 
- [Hel] W. Helbig. *Praxiswissen in der Messtechnik. Arbeitsbuch für Techniker, Ingenieure und Studenten.*
- [Her+12] E. Hering u. a. *Elektrotechnik und Elektronik für Maschinenbauer.* Bd. 2. Springer, 14. Feb. 2012. ISBN: 978-3-642-12881-3.
- [Joy] *ARKADE-BUTTONS MIT MIKROSCHALTER.* 2024.
- [Rs2] *Drucktaster von Eaton: Ein Leitfaden.* 2026. URL: <https://de.rs-online.com/web/content/discovery-portal/produktanbieter/drucktaster-leitfaden> (besucht am 09.05.2026).
- [Smi11] A. G. Smith. *Introduction to Arduino: A Piece of Cake!* Self-published, 2011. ISBN: 978-1463698348.
- [TR14] H.-R. Tränkler und L. Reindl, Hrsg. *Sensortechnik: Handbuch für Praxis und Wissenschaft.* 2. Aufl. VDI-Buch. Published: 13 January 2015. eBook Packages: Computer Science and Engineering (German Language). Berlin, Heidelberg: Springer Vieweg Berlin, Heidelberg, 2014, S. XIV, 1596. ISBN: 978-3-642-29942-1. DOI: **10.1007/978-3-642-29942-1**.
- [Zie25] D. Zielke. *Mikrosysteme.* Bd. 1. Springer, 5. März 2025. ISBN: 978-3-662-71232-0.